Marcelo Fernandes de Oliveira Junior
Gustavo Oliveira dos Santos
Christian Lima Santana

Análise Comparativa de Desempenho e Eficiência Energética entre Processamento de OCR em Borda e em Nuvem aplicado à Smart Metering.

São Paulo
2026

Marcelo Fernandes de Oliveira Junior
Gustavo Oliveira dos Santos
Christian Lima Santana

Análise Comparativa de Desempenho e Eficiência Energética entre Processamento de OCR em Borda e em Nuvem aplicado à Smart Metering.

Trabalho de Conclusão de Curso II apresentado ao SENAC Santo Amaro como requisito parcial para conclusão do curso de Bacharelado em Ciência da Computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Curso de Bacharelado em Ciência da Computação

Orientador: Afonso César Lelis Brandão

São Paulo
2026

Resumo

Este trabalho busca propor uma análise comparativa entre dois métodos de processamento para sistemas de Smart Metering (medição inteligente): Edge Computing (Computação em Borda), com inferência de Optical Character Recognition (OCR) realizada localmente no microcontrolador ESP32-CAM por meio de modelo de aprendizado de máquina treinado de forma supervisionada e quantizado em inteiros de 8 bits, e Cloud Computing (Computação em Nuvem), com inferência realizada em ambiente remoto através de modelo treinado de forma supervisionada e quantizado em ponto flutuante de 32 bits. A motivação reside no elevado custo de substituição dos mais diversos medidores convencionais, amplamente utilizados em ambientes residenciais e privados, por modelos inteligentes nativos, o que favorece o *retrofitting* como alternativa mais economicamente viável. A transmissão dos dados entre os nós de borda e o gateway central é realizada via rede Long Range Wide Area Network (LoRaWan), utilizando o Spreading Factor SF8, configuração que assegura conformidade com o limite regulatório de *dwell time* de 400 ms estabelecido pelo Ato nº 14.448 da ANATEL, ao mesmo tempo em que maximiza o alcance dentro desse limite. A viabilidade do envio de imagens fragmentadas via LoRaWan foi demonstrada analiticamente, com uma imagem de 160×120 pixels em escala de cinza, comprimida utilizando o algoritmo JPEG com aproximadamente 3,10 KB, podendo ser transmitida em 38 pacotes em cerca de 13,3 s de rádio ativo. Os dois fluxos de processamento serão avaliados segundo três métricas quantitativas: acurácia de leitura OCR (%), latência de ponta a ponta (ms) e consumo energético por leitura (mAh). O objetivo deste trabalho é apresentar resultados que orientem a escolha arquitetural mais adequada para implantações de *Smart Metering* em larga escala, considerando o *trade-off* entre eficiência energética, precisão de reconhecimento e complexidade de manutenção do sistema.

Palavras-chave: Smart Metering. ESP32. LoRaWan. OCR.

Abstract

This work aims to propose a comparative analysis between two processing methods for Smart Metering systems: Edge Computing, with OCR inference performed locally on the ESP32-CAM microcontroller using a machine learning model trained in a supervised manner and quantized to 8-bit integers, and Cloud Computing, with inference performed in a remote environment through a model trained in a supervised manner and quantized to 32-bit floating point. The motivation lies in the high replacement cost of the most diverse conventional meters, widely used in residential and private environments, by native smart models. This favors *retrofitting* as a more economically viable alternative. Data transmission between the edge nodes and the central gateway is carried out via a LoRaWan network, using Spreading Factor SF8, a configuration that ensures compliance with the dwell time regulatory limit of 400 ms established by ANATEL Act No. 14.448, while maximizing range within that limit. The feasibility of sending fragmented images via LoRaWan was demonstrated analytically, with a 160×120 pixel grayscale image, compressed using the JPEG algorithm to approximately 3.10 KB, which can be transmitted in 38 packets in about 13.3 s of active radio time. The two processing flows will be evaluated according to three quantitative metrics: OCR reading accuracy (%), end-to-end latency (ms), energy consumption per reading (mAh). The objective of this work is to present results that guide the most appropriate architectural choice for large-scale *Smart Metering* deployments, considering the *trade-off* between energy efficiency, recognition precision, and system maintenance complexity. **Keywords**: Smart Metering. ESP32. LoRaWan. OCR.

Lista de ilustrações

Figura 1 – Campos contidos em um pacote de Uplink 17
Figura 2 – Campos contidos em Frame Header em Downlink 17
Figura 3 – Pilha de elementos das camadas LoRa/LoRaWan 19
Figura 4 – Implementação Típica de Rede LoRaWan 19
Figura 5 – Módulo ESP32-CAM integrado com LoRa 21
Figura 6 – Distribuição de Trabalhos Relacionados à Computação em IoT 23
Figura 7 – Arquitetura do sistema e os dois fluxos de processamento 40
Figura 8 – Evolução do treino do modelo de localização da janela 51
Figura 9 – Evolução do treino do classificador de dígitos (fase A) 52
Figura 10 – Leitura com a janela de dígitos marcada manualmente em cinco capturas do ESP32-CAM 54

Lista de tabelas

Tabela 1 – Strings de busca utilizadas no levantamento bibliográfico. 15
Tabela 2 – Classes de Configuração LoRa 16
Tabela 3 – Parâmetros de Configuração LoRa 18
Tabela 4 – Vantagens e Desvantagens na Adoção de Métodos de Processamento . 24
Tabela 5 – Uplink: Canais de Frequência 26
Tabela 6 – Downlink: Canais de Frequência 26
Tabela 7 – Taxas de transmissão Uplink e configurações LoRaWAN. 27
Tabela 8 – Parâmetros de Downlink Data Rate 27
Tabela 9 – Downlink RX1 Data Rate Mapping 28
Tabela 10 – Componentes de hardware do protótipo 41
Tabela 11 – Principais componentes de software 42
Tabela 12 – Parâmetros do enlace LoRaWAN 44
Tabela 13 – Formato do *payload* de aplicação em cada fluxo 45
Tabela 14 – Tamanho médio do JPEG de três imagens de 160×120 pixels 45
Tabela 15 – Pacotes, tempo de rádio e tempo total de envio por leitura 47
Tabela 16 – Pipelines de OCR desenvolvidos 49
Tabela 17 – Arquitetura do classificador de dígitos 52
Tabela 18 – Comparação de métricas reportadas 55

Lista de Abreviaturas e Siglas

ABP Activation by Personalization 20, 40, 42, 44

BW Bandwidth 18, 44

CNN Convolutional Neural Network 31, 34

CR Coding Rate 18, 44

CRNN Convolutional Recurrent Neural Network 33

CSS Chirp Spread Spectrum 26

CTC Connectionist Temporal Classification 34

DR Data Rate 26, 27

EIRP Effective Isotropic Radiated Power 28

IA Inteligência Artificial 28, 29

IaaS Infrastructure as a Service 23

IoT Internet of Things 11–14, 19, 22, 23

ISM Industrial, Scientific and Medical 24, 25

LoRa Long Range 12, 14, 16, 25, 26, 28, 37, 43, 46

LoRaWan Long Range Wide Area Network 2, 3, 13–16, 18, 19, 36, 38–42, 44, 47

LPWAN Low Power Wide Area Network 18

LR-FHSS Long Range Frequency Hopping Spread Spectrum 26

OCR Optical Character Recognition 2, 3, 5, 12, 13, 28, 32, 35, 37–42, 44, 47–49

OTAA Over the Air Activation 20

PDR Packet Delivery Ratio 38

QAT Quantization-Aware Training 35

RGB Red, Green, Blue 31

RNA Redes Neurais Artificiais 35

RNN Recurrent Neural Network 34

ROI Region of Interest 33

SaaS Software as a Service 23

SF Spreading Factor 18, 26, 43, 44

Sumário

1 INTRODUÇÃO . . . 11
1.1 Contextualização do problema . . . 11
1.2 Formulação do problema . . . 12
1.3 Objetivos e hipótese . . . 12
1.3.1 Objetivo geral . . . 12
1.3.2 Objetivos específicos . . . 13
1.3.3 Hipótese . . . 13
1.4 Justificativa . . . 13

2 REVISÃO BIBLIOGRÁFICA / TRABALHOS RELACIONADOS . . 15
2.1 Metodologia da pesquisa bibliográfica . . . 15
2.2 Fundamentos teóricos . . . 16
2.2.1 LoRa . . . 16
2.2.2 LoRaWan . . . 18
2.2.2.1 Dispositivos Finais (End Nodes) . . . 19
2.2.2.2 Gateways . . . 21
2.2.2.3 Servidores de Rede e Aplicação . . . 22
2.2.3 Edge Computing e Cloud Computing . . . 23
2.2.4 Restrições Regulatórias . . . 24
2.2.5 Inteligência Artificial . . . 28
2.2.5.1 Machine Learning . . . 28
2.2.5.1.1 Aprendizado Supervisionado . . . 29
2.2.5.1.2 Deep Learning . . . 29
2.2.5.1.3 O que são LSTMs. . . . 29
2.2.5.1.4 Limitações em *Edge AI* e alternativas convolucionais. . . . 30
2.2.5.2 Redes Neurais . . . 30
2.2.5.2.1 Redes Neurais Convolucionais . . . 30
2.2.5.2.2 Camadas que Constituem uma CNN . . . 31
2.2.5.2.3 Camada Convolucional . . . 31
2.2.5.2.4 Camada de Pooling . . . 31
2.2.5.2.5 Camada Totalmente Conectada . . . 32
2.2.5.3 OCR - Fundamentos e Escopo . . . 32
2.2.5.3.1 Pipeline Tradicional . . . 32
2.2.5.3.2 Evolução Tecnológica: dos Métodos Heurísticos ao Deep Learning . . . 33
2.2.5.4 Arquitetura U-Net - Segmentação de ROI . . . 33

2.2.5.5 Arquitetura CRNN e CTC . . . 33
2.2.5.6 Quantização e QAT . . . 35
2.3 Trabalhos correlatos . . . 35
2.3.1 Trabalho 1: Solução IoT para Digitalização de Medidores de Consumo por Visão Computacional e MicroML . . . 35
2.3.2 Trabalho 2: Sistema de Gerenciamento Inteligente de Água Utilizando Sensores Baseados em IoT . . . 36
2.3.3 Trabalho 3: Rede de Sensores Sem Fio Habilitada para LoRa para Leitura e Faturamento de Consumo de Água Usando Reconhecimento Óptico de Caracteres . . . 37

3 DESENVOLVIMENTO / IMPLEMENTAÇÃO / EXPERIMENTOS 38
3.1 Metodologia da pesquisa . . . 38
3.1.1 Desempenho da Rede de Comunicação . . . 38
3.1.2 Acurácia da Visão Computacional (OCR) . . . 38
3.1.3 Autonomia Energética . . . 39
3.2 Detalhes da implementação . . . 39
3.2.1 Visão geral da solução . . . 39
3.2.2 Componentes de hardware . . . 40
3.2.3 Componentes de software . . . 41
3.2.4 Simplificações em relação a uma implantação real . . . 42
3.2.5 Enlace LoRaWAN . . . 43
3.2.5.1 Estudo preliminar e escolha da configuração de rádio . . . 43
3.2.5.2 Formato das mensagens . . . 44
3.2.5.3 Preparo da imagem e fragmentação . . . 45
3.2.5.4 Tempo no ar . . . 46
3.2.5.5 Recepção, descriptografia e remontagem . . . 47
3.2.6 Reconhecimento dos dígitos . . . 48
3.2.6.1 Evolução das abordagens . . . 48
3.2.6.1.1 Teste de viabilidade com OCR genérico. . . . 48
3.2.6.1.2 Pipelines avaliados. . . . 48
3.2.6.2 Conjuntos de dados . . . 49
3.2.6.3 Modelo de localização da janela . . . 50
3.2.6.4 Classificador de dígitos . . . 51
3.2.6.5 Quantização e exportação . . . 53
3.2.6.6 Leitura com janela calibrada na instalação . . . 53
3.3 Experimentos e cenário de testes (opcional) . . . 54
3.4 Resultados e análises . . . 54

4 CONCLUSÃO . . . 56

4.1 Retomada do problema e dos objetivos 56
4.2 Contribuições para a Computação 56
4.3 Limitações e trabalhos futuros 56

Observações importantes 57

Apresentação oral (banca) 58

REFERÊNCIAS 60

1 Introdução

Atualmente, a implantação de conceitos de Internet of Things (IoT) nos mais diferentes dispositivos presentes no dia a dia de milhões de pessoas ao redor do mundo está numa crescente jamais antes vista. Como explicado por Tedjojuwono e Jahja (2024), a adoção de dispositivos IoT é um dos principais pilares na ascensão da indústria 4.0, principalmente na pauta da construção de cidades inteligentes. E ainda, por característica, tais tecnologias viabilizam a coleta de dados de forma constante, promovendo insumos para realização de análises, manutenções e aprimoramentos de sistemas através de planos de ação mais eficientes e eficazes.

Em contrapartida, como explicado por Fortune Business Insights (2025) e Uggeri e Simeon (2020), o alto custo inicial exigido para realizar a transição de dispositivos analógicos para soluções inteligentes nativas constitui o principal entrave para a adoção de tais tecnologias em larga escala. A complexidade de infraestrutura necessária para a substituição física dos medidores limita a viabilidade econômica de projetos em áreas privadas de pequeno e médio porte.

Abordando a pauta de automação residencial e comercial, o tema de smart metering surge como ponto focal na discussão. Como evidenciado pelo contraste em abordagens de Vilela (2021) e Adam (2022), o desenvolvimento de sistemas focados em smart metering enfrenta um impasse arquitetônico, onde o método de processamento divergia entre os projetos.

A implementação proposta por Vilela (2021) é a denominada Edge Computing, onde o processamento é realizado localmente no dispositivo final, assim otimizando recursos energéticos e latência. Em oposição, a proposta de Adam (2022), realizando processamento em cloud (Cloud Computing), surge como uma solução focada em facilidade de manutenção, utilização de softwares mais robustos e menor complexidade de implementação.

Neste contexto, a finalidade deste trabalho é realizar a comparação entre os métodos de processamento citados, definindo limites, vantagens e desvantagens sob determinado ambiente.

1.1 Contextualização do problema

O mundo, neste momento, está passando pelo que pode ser chamado de era da digitalização, havendo severas transformações no modo de transmissão de dados e informações (TEDJOJUWONO; JAHJA, 2024). Atualmente, a humanidade encontra-se sob influência da expansão da denominada indústria 4.0, que tem como foco a digitalização dos processos de manufatura e produção, mas que engloba inclusive o modo de vida e acesso a informações da sociedade sob o braço tecnológico denominado *Cidades Inteligentes*.

Neste contexto, a utilização de hardware inteligente deixou de ser mero luxo de grandes corporações para se tornar o paradigma central na mudança citada anteriormente. Em relação a equipamentos de medição inteligentes, também classificados como Smart Metering, hoje já há uma série de avanços que possibilitam o processamento de dados de maneira mais veloz e eficiente. Mas essa evolução também acompanhou a necessidade de adaptação de infraestrutura, assim agregando altos custos para a transição das tecnologias analógicas para o digital (ALVISI et al., 2019).

Como apresentado por Alvisi et al. (2019), o retrofitting é uma das soluções mais difundidas para contornar o alto custo de digitalização de sistemas nativamente analógicos. Tal conceito tem como objetivo realizar a digitalização de sistemas analógicos reaproveitando o hardware já existente. Essa concepção é amplamente abordada na academia, havendo grandes teses, trabalhos e dissertações sobre o tema.

Diante de tal abordagem para redução de custos, a aplicabilidade da técnica de retrofitting enfrenta um impasse. Atualmente, na academia encontram-se mais difundidos trabalhos que abordam o uso de microcontroladores aliados à tecnologia Long Range (LoRa) utilizando o método de processamento em borda. Como explicado por Vilela (2021), a utilização deste método traz grandes vantagens em relação à latência e otimização de consumo energético. Em oposição, como Andriulo et al. (2024) apresenta, o método de processamento em cloud destaca-se por proporcionar maior granularidade de dados, simplificação no desenvolvimento de firmware e acurácia ligeiramente melhor em relação ao conceito de Edge Computing. Todavia, permanece a dúvida de qual método de processamento utilizar no desenvolvimento de uma solução IoT baseada na técnica de retrofitting. A resolução do problema criará um alicerce, auxiliando o desenvolvimento de futuras ferramentas que, por concepção, apresentarão a mesma natureza de problemática.

1.2 Formulação do problema

Coloque o problema em forma de pergunta clara, delimitando o que será investigado e qual resultado se espera atingir dentro da Computação. A pergunta de pesquisa deve ser específica o suficiente para ser respondida ao longo do trabalho e diretamente derivada da lacuna apresentada na contextualização.

1.3 Objetivos e hipótese

1.3.1 Objetivo geral

Comparar a viabilidade técnica, a acurácia e o consumo energético entre o processamento de OCR realizado localmente no ESP32-CAM (abordagem Edge Computing) e a inferência

OCR realizada remotamente em servidor na nuvem (Cloud), utilizando a rede LoRaWan como meio de transmissão.

1.3.2 Objetivos específicos

1. Implementar os dois fluxos de processamento que compõem o experimento comparativo: o fluxo Edge, em que o OCR ocorre localmente no ESP32-CAM; e o fluxo Cloud, em que a imagem é transmitida para processamento remoto;

2. Configurar a rede LoRaWan conectando os nós da rede com o objetivo de reduzir ao máximo o consumo energético;

3. Avaliar e comparar os dois fluxos segundo as métricas: precisão de leitura (%), latência de processamento (ms), consumo energético por leitura (mAh) e volume de dados transmitidos (bytes/leitura), identificando o *trade-off* entre processamento local e transmissão.

1.3.3 Hipótese

Considerando o trade-off apresentado, foram estabelecidas hipóteses com o objetivo de verificar se a utilização de determinada abordagem apresenta vantagens quando observadas sob o mesmo cenário. As hipóteses orientam a definição dos experimentos e a análise dos resultados obtidos.

- **H1**: A quantização em inteiros de 8 bits, aplicada ao modelo implementado no fluxo de processamento em borda, causa redução de acurácia em comparação ao modelo quantizado em ponto flutuante de 32 bits, implementado no fluxo de processamento em nuvem.

- **H2**: O método de processamento local apresenta menor latência e maior valor de taxa de pacotes entregues, em comparação com o método de processamento remoto.

- **H3**: A abordagem de processamento em nuvem apresenta autonomia energética inferior durante a transmissão de mensagens quando comparada ao método de processamento local.

1.4 Justificativa

Este trabalho propõe uma abordagem analítica sobre métodos de processamento implementados em sistemas baseados em tecnologias IoT com o objetivo de aplicação da técnica de retrofitting em instrumentos de medição analógicos. Tal temática tem como alicerce a lacuna literária identificada por Andriulo et al. (2024), na qual carecem trabalhos

que apresentam informações úteis para a etapa de desenvolvimento arquitetural de tais ferramentas.

Diante de tal contexto, revela-se a importância acadêmica deste trabalho, apresentando de forma científica comparações entre os métodos de processamento, definindo limites e estipulando vantagens na adoção de determinado método. E ainda, este trabalho tem como pauta discorrer sobre os temas: desenvolvimento de sistemas baseados em tecnologia LoRa, aplicação de algoritmos de packet loss (perda de pacotes) e uplink de dados segmentados em pacotes via LoRaWan.

O desenvolvimento deste trabalho tem por foco a disponibilização de insumos para tomada de decisões no desenvolvimento de sistemas IoT que empregam protocolos de rede de baixa largura de banda.

2 Revisão Bibliográfica / Trabalhos Relacionados

A revisão bibliográfica tem como finalidade apresentar o embasamento teórico do trabalho, demonstrando o conhecimento do autor sobre o tema e identificando o estado da arte. Este capítulo está organizado em três seções: a **metodologia da pesquisa bibliográfica** (como as fontes foram levantadas e selecionadas), os **fundamentos teóricos** (os conceitos, teorias e técnicas que fundamentam a proposta) e os **trabalhos correlatos** (estudos que enfrentaram o mesmo problema, com suas limitações).

2.1 Metodologia da pesquisa bibliográfica

A pesquisa bibliográfica foi conduzida de forma sistemática, estruturando-se em três temas principais: Hardware, Software e Redes. Para a seleção dos trabalhos, utilizaram-se as bases de dados Google Scholar, MDPI e Intechopen.

O processo de seleção seguiu um fluxo de filtragem (funil). Inicialmente, foram identificados 120 artigos por meio da análise de títulos, utilizando strings de busca majoritariamente em inglês, tais como “iot AND energy efficiency” e “ESP32 AND camera monitoring”. Como ferramentas auxiliares na descoberta de conexões entre autores e na triagem inicial de relevância, foram utilizadas as plataformas ResearchRabbit e Gemini Deep Research.

Na etapa de refinamento, aplicou-se a leitura dos resumos (abstracts) e a verificação de disponibilidade integral dos textos. Foram adotados como critérios de inclusão: artigos publicados entre 2020 e 2026, relevância direta para sistemas de telemetria e foco em implementações práticas. Como critério de exclusão, descartaram-se trabalhos com acesso restrito ou que não apresentassem detalhes técnicos sobre a integração dos eixos propostos.

Tabela 1 – Strings de busca utilizadas no levantamento bibliográfico.

ID	Eixo Temático	String de Busca
ST01	Redes	"IoT"AND "LoRaWan"AND "water waste"
ST02	Contexto	smart water meter market
ST03	Contexto	smart water meter costs and benefits
ST04	Hardware	"IoT"AND "energy efficiency"
ST05	Hardware	"ESP32"AND "camera monitoring"
ST06	Hardware	"ESP32-CAM"AND "analog meter"AND "OCR"
ST08	Software	"automatic meter reading"OCR accuracy "error rate"evaluation
ST09	Redes	"packet delivery ratio"LoRaWan "latency"monitoring system
ST010	Contexto	"water consumption predictionmean absolute error"forecasting metrics

2.2 Fundamentos teóricos

2.2.1 LoRa

Como explicado por Pastório et al. (2022), LoRa é uma tecnologia, patenteada e desenvolvida pela Semtech, que tem como foco viabilizar a transmissão de dados sem fio em longo alcance através de radiofrequência.

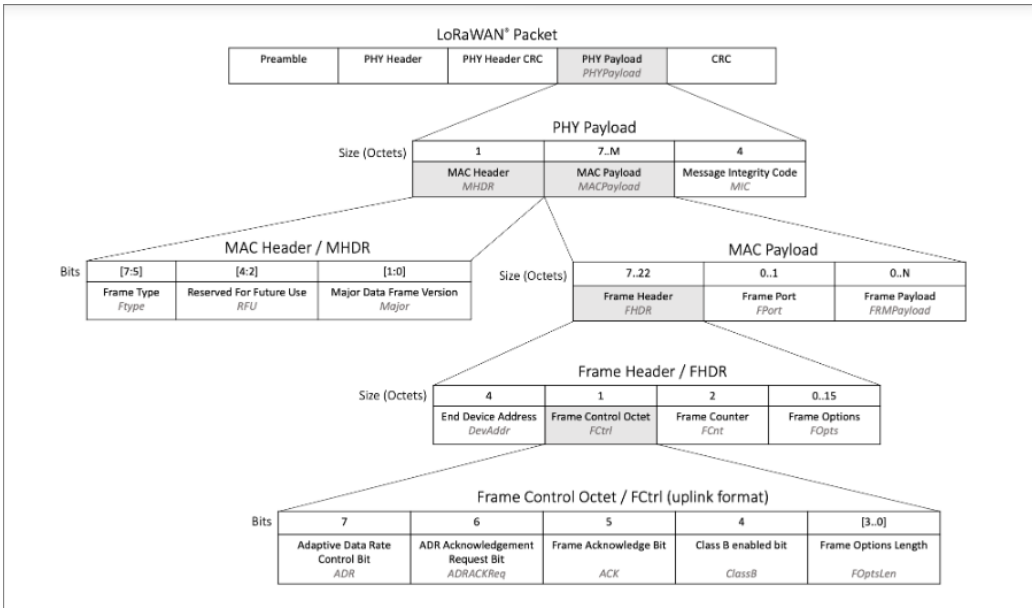
A rede LoRaWan é constituída por dispositivos finais, gateways e servidores de rede/aplicação. Com o objetivo de realizar um gerenciamento mais fino de consumo energético, a comunicação entre os dispositivos finais e servidores de rede é definida por três classes como demonstrado na tabela 2.

Tabela 2 – Classes de Configuração LoRa

Classe	**Descrição**	**Característica**
Classe A	O dispositivo final é habilitado a receber downlink imediatamente após ser realizado um envio de mensagem (uplink).	Menor consumo energético e menor tráfego de mensagens.
Classe B	O dispositivo final é habilitado a receber downlink em períodos agendados e conhecidos.	Consumo energético significativamente maior que a classe A, mas viabiliza comunicação mais frequente entre servidor e dispositivo final.
Classe C	O dispositivo final é continuamente habilitado a receber downlink.	Consumo energético muito alto, mas viabiliza a comunicação entre as partes de forma quase ininterrupta.

Fonte: Adaptado de Semtech Corporation (Semtech Corporation, 2024b).

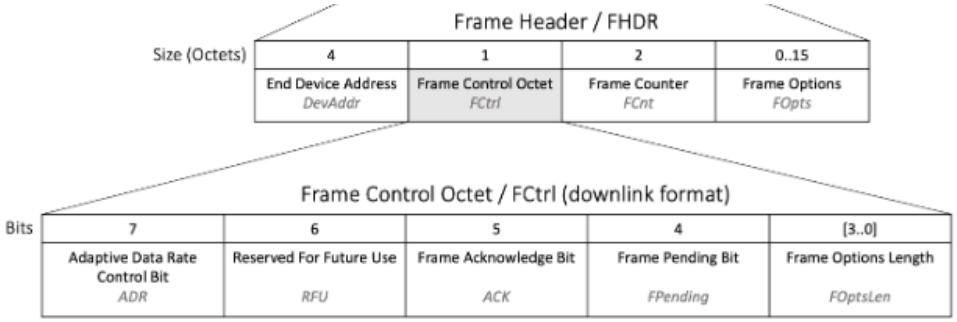
Figura 1 – Campos contidos em um pacote de Uplink



Fonte: Adaptado de Semtech Corporation (Semtech Corporation, 2024c).

A figura 1 apresenta a estrutura de pacote de rede que é utilizada na tecnologia LoRa. Tal estrutura é utilizada tanto no envio de dados pelo dispositivo final (uplink) quanto no recebimento de dados (downlink).

Figura 2 – Campos contidos em Frame Header em Downlink



Fonte: Adaptado de Semtech Corporation (Semtech Corporation, 2024c).

A estrutura do pacote de rede de uplink e downlink é a mesma, mas há diferenças em flags específicas entre os tipos de transmissão de dados. A figura 2 apresenta a estrutura do "Frame Header / FHDR" utilizada em downlink. As estruturas de pacote divergem na flag de classe B, no uplink, por meio da qual o dispositivo final informa ao servidor estar no modo de recebimento de mensagens em tempo programado, e, no pacote downlink, na flag FPending, utilizada para informar a existência de mais mensagens a serem enviadas pelo servidor/gateway.

Conforme apresentado pela LoRa Alliance Regional Parameters Workgroup (2025) o tamanho máximo de MACPayload, respeitando o padrão regulatório AU915, que será mais bem explicado e abordado posteriormente, em bytes, é variável e dependente do tamanho ocupado pelo atributo Fopts, que é utilizado para solicitar alterações de parâmetros de rede ao dispositivo final. Não utilizando repetidores de sinal que interligam dispositivos finais com gateways, o tamanho máximo de MACPayload é de 250 bytes, restando em torno de 227 bytes utilizáveis em payload.

Para viabilizar a modulação LoRa em dispositivo final, é necessário realizar a configuração de três parâmetros fundamentais. A tabela 3 apresenta as definições dos principais parâmetros que ajustam o funcionamento da rede LoRaWan.

Tabela 3 – Parâmetros de Configuração LoRa

Parâmetro	**Definição**
Bandwidth (BW)	Determina a capacidade máxima de transmissão em determinado período de tempo (medido em kHz).
Spreading Factor (SF)	Determina o alcance do sinal de rádio; o SF é determinado por classes, nas quais, quanto maior a classe, maior a área de alcance do sinal.
Coding Rate (CR)	Número de bits destinados à redundância de dados, utilizado para possibilitar recuperação da mensagem em caso de erros na transmissão.

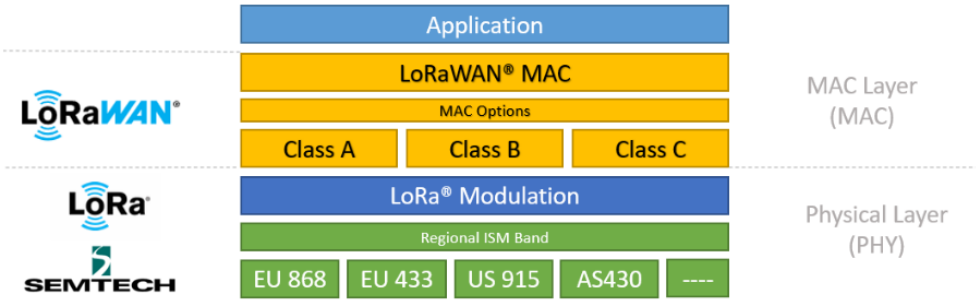
Fonte: Adaptado de Pastório et al. (2022).

2.2.2 LoRaWan

O LoRaWan é um protocolo de comunicação baseado na tecnologia Low Power Wide Area Network (LPWAN), que possui o objetivo de viabilizar a comunicação em grandes distâncias, conciliada ao baixo consumo energético. O protocolo é open-source e mantido pela LoRa Alliance. Como citado anteriormente, a rede LoRaWan é formada por dispositivos finais, gateways, servidores de rede e servidores de aplicação (PASTóRIO et al., 2022).

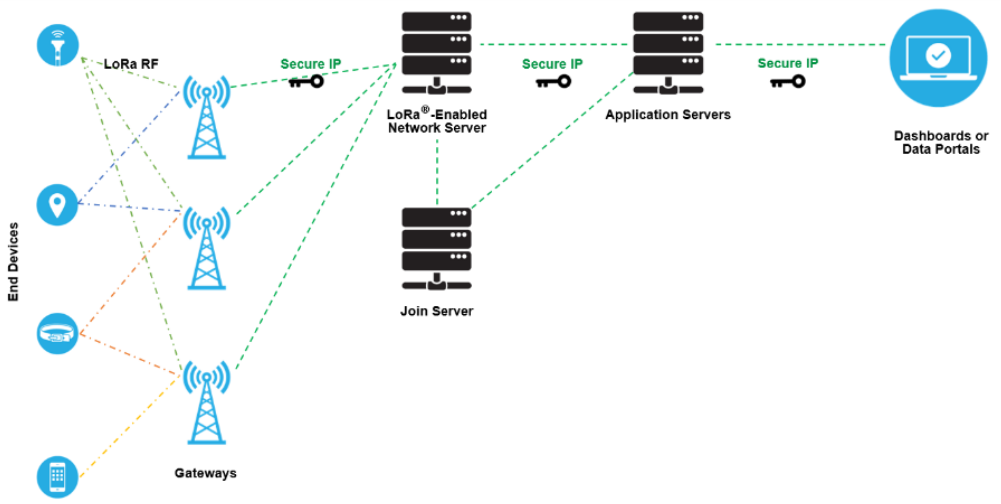
A figura 3 apresenta a pilha de elementos das camadas MAC e Física que compõem a arquitetura do protocolo LoRaWan.

Figura 3 – Pilha de elementos das camadas LoRa/LoRaWan



Fonte: Adaptado de Semtech Corporation (Semtech Corporation, 2024a).

Figura 4 – Implementação Típica de Rede LoRaWan



Fonte: Adaptado de Semtech Corporation (Semtech Corporation, 2024a).

A figura 4 apresenta a arquitetura típica de um sistema IoT que utiliza como base o protocolo LoRaWan para viabilizar a comunicação entre os dispositivos. Cada elemento que compõe uma rede LoRaWan possui uma responsabilidade; neste momento, é extremamente válido analisá-las individualmente:

2.2.2.1 Dispositivos Finais (End Nodes)

Semtech Corporation (2024a) define que dispositivos finais habilitados para LoRaWan são sensores ou atuadores que são conectados a gateways via modulação de radiofrequência LoRa.

Na maioria das aplicações, esses dispositivos são autônomos, dependentes de baterias como fonte energética primária e possuem como responsabilidade digitalizar condições físicas como temperatura, captura de imagens, níveis de tremulação, entre outros.

Em relação às possíveis configurações que podem ser aplicadas nos dispositivos finais, como apresentado por Pastório et al. (2022), cada nó pode assumir uma das configurações que a tabela 2 apresenta. Outro parâmetro configurável é o tipo de autenticação dos dispositivos em rede, havendo o padrão Over the Air Activation (OTAA), no qual o próprio dispositivo gerencia a conexão à rede, e a Activation by Personalization (ABP), na qual os dispositivos são vinculados de forma manual a uma rede em específico, não passando pelo processo de entrada implementado no OTAA.

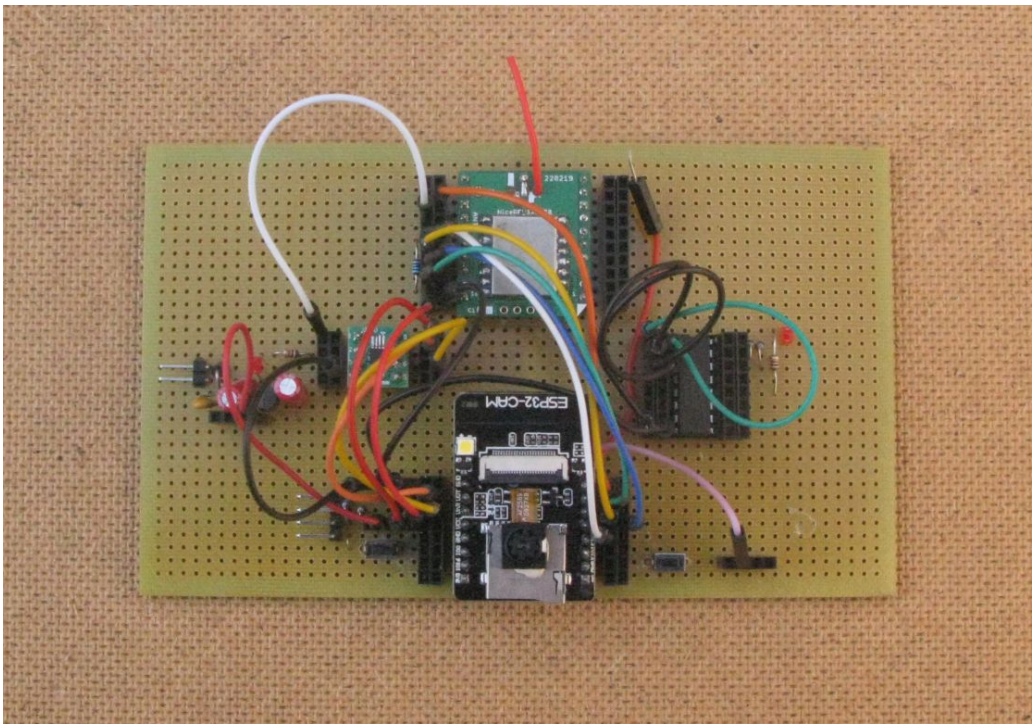
O LoRaWan utiliza o padrão AES de criptografia simétrica, aplicado tanto na camada de aplicação quanto na camada de rede.

Os dispositivos finais, corriqueiramente, são desenvolvidos sobre microprocessadores, podendo tanto utilizar placas que já incluem módulos LoRaWan nativos quanto serem desenvolvidos sobre plataformas como ESP32, Arduino, Raspberry Pi, entre outras. A partir da construção do dispositivo, ou acoplamento de módulos, possibilita-se a captura de dados em ambiente (PASTóRIO et al., 2022).

Como destacado por Sari et al. (2022) e Ahmad et al. (2021), o ESP32 destaca-se nesse cenário como uma plataforma robusta de baixo custo, integrando conectividade sem fio e múltiplos modos de operação. Diferentemente de arquiteturas mais simples, o ESP32 permite a execução de tarefas complexas e segurança criptográfica sem comprometer severamente o ciclo de vida da bateria, desde que devidamente configurado.

Segundo Ferreira e Cavalcante (2020), a vida útil da bateria é influenciada criticamente não apenas pelo hardware, mas pela eficiência das rotinas de escrita em periféricos e pela otimização do firmware. Diante de tal afirmação, destaca-se a função *deep sleep* presente no ESP32 que possui como foco a minimização do consumo energético quando em estado ocioso.

Figura 5 – Módulo ESP32-CAM integrado com LoRa



Fonte: (ROBINSON, 2021).

2.2.2.2 Gateways

O gateway, como demonstrado na figura 4, é um elemento da rede LoRaWan que tem como responsabilidade agrupar dispositivos finais e viabilizar a comunicação entre o dispositivo final e o servidor de rede (PASTóRIO et al., 2022).

A comunicação entre o dispositivo final e o gateway é realizada via protocolo LoRaWan, na qual o gateway pode realizar a tarefa de registrar tráfego de mensagens. Como explicado por Semtech Corporation (2024a), os dispositivos finais, ao realizar o envio de uplink, distribuem as suas mensagens a todos os gateways alcançáveis dentro da mesma rede; tal função tem como objetivo garantir que ao menos um dos gateways da rede receba e transmita a mensagem ao servidor de rede.

Os gateways operam na camada física e, convencionalmente, assumem a responsabilidade apenas de viabilizar o tráfego de requisições uplink e downlink e realizar a verificação de integridade do pacote, não realizando processamento intermediário da mensagem para enviá-la ao servidor de rede (Semtech Corporation, 2024a).

A comunicação entre o gateway e o servidor de rede, corriqueiramente, é estabelecida utilizando protocolo TCP/IP. Toda comunicação dentro do protocolo LoRaWan é bidirecional (PASTóRIO et al., 2022).

É possível encontrar gateways prontos de múltiplos canais sendo comercializados, mas é plenamente viável desenvolver gateways a partir de módulos e placas de prototipagem

como a apresentada na figura 5.

2.2.2.3 Servidores de Rede e Aplicação

Como apresentado por Pastório et al. (2022), servidores de rede e servidores de aplicação, dada a estrutura em que foram desenvolvidos, tanto podem ser implementados sob o mesmo hardware, como podem estar fisicamente distantes, realizando comunicação através de protocolo TCP/IP.

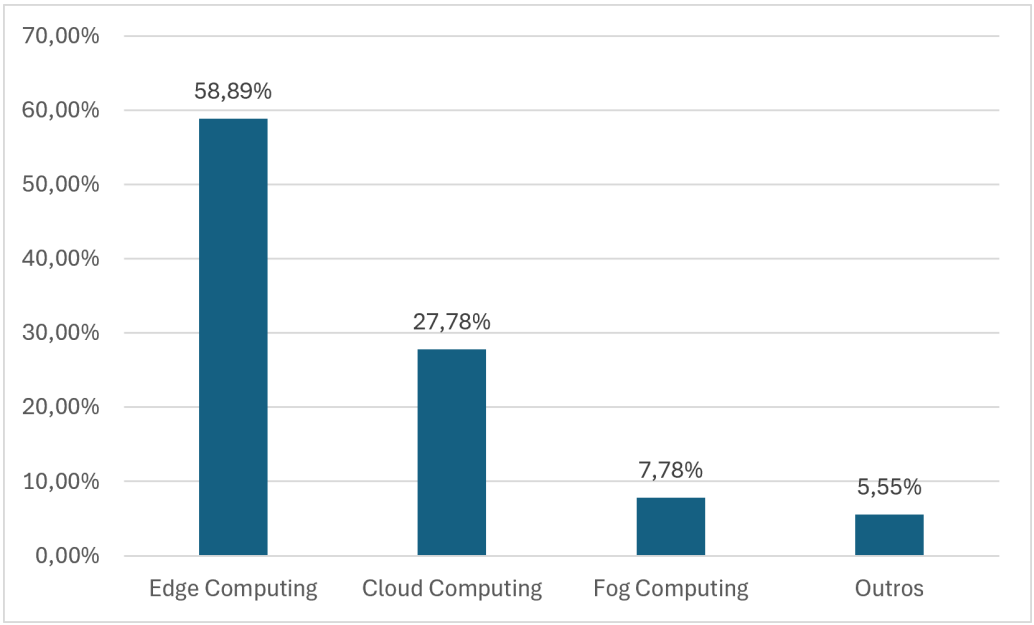
Como explicado pela Semtech Corporation (2024a), servidores de rede são responsáveis por gerenciar dinamicamente a rede de dispositivos, adaptando o sistema para se ajustar a um cenário de constante mudança de parâmetros. Entre as responsabilidades dos servidores de rede estão: receber, descartar duplicatas e descriptografar mensagens enviadas por uplink, realizar autenticação, garantir segurança com o protocolo AES, confirmar recebimento de mensagens, responder a requisições de camada MAC, realizar configuração de dispositivos finais através de requisições downlink, entre outras.

Servidores de Aplicação, por sua vez, são responsáveis por receber pacotes enviados pelo servidor de rede, tratar e disponibilizar em formato de informação ao usuário os dados extraídos pelos dispositivos finais. Como a Semtech Corporation (2024a) explica, é uma de suas responsabilidades também gerenciar os payloads de downlink a serem enviados aos dispositivos finais.

Ambos os tipos de servidores possuem diversas soluções prontas para implementação tanto em ambiente local quanto em nuvem. Para servidores de rede, destaca-se o Chirp Stack, que tem por foco o gerenciamento de dispositivos de IoT em rede privada. Já para servidores de aplicação, o Tago IO mostra-se como uma ferramenta robusta, com foco corporativo e ampla gama de ferramentas voltadas para gerenciamento de dispositivos (PASTóRIO et al., 2022).

2.2.3 Edge Computing e Cloud Computing

Figura 6 – Distribuição de Trabalhos Relacionados à Computação em IoT



Fonte: Adaptado de Andriulo et al. (2024).

Edge Computing (computação em borda), em IoT, é uma arquitetura baseada em processamento descentralizado de dados, na qual ocorre o processamento com o objetivo de construção de informações a partir de dispositivos que têm por responsabilidade inicial a extração de dados do ambiente real. Como explicado por Andriulo et al. (2024), o gráfico apresentado na figura 6 demonstra que Edge Computing é a abordagem mais amplamente difundida na literatura, sendo uma solução voltada à segurança de informações e à otimização de redes de baixa largura de banda, promovendo possível redução de custos operacionais.

Cloud Computing (computação em nuvem), ainda no ambiente IoT, é um tipo de arquitetura pautado pela alta escalabilidade de armazenamento de dados e poder computacional centralizado ou não. Como explicado também por Andriulo et al. (2024), este tipo de arquitetura tem como prós a possível adoção de provedores de nuvem, acoplando serviços como Infrastructure as a Service (IaaS), Software as a Service (SaaS), entre outros, assim reduzindo investimentos iniciais, apresentando maior facilidade de manutenção sistêmica e menor complexidade no desenvolvimento de firmware.

Tabela 4 – Vantagens e Desvantagens na Adoção de Métodos de Processamento

Método	**Vantagens**	**Desvantagens**
Edge Computing	• Menor latência na transmissão de mensagens; • Menor tráfego de mensagens em rede; • Foco em segurança de dados;	• Possíveis altos custos iniciais; • Capacidade computacional limitada; • Dificuldades de execução de modelos de machine learning; • Dificuldades em escalabilidade;
Cloud Computing	• Maior poder computacional disponível; • Escalabilidade; • Possível menor custo inicial;	• Alta latência; • Maior risco em segurança de dados; • Maior quantidade de pacotes enviados à rede;

Fonte: Adaptado de Andriulo et al. (2024).

A tabela 4 explicita que tanto Cloud Computing quanto Edge Computing trazem prós e contras em relação à sua implementação em projetos. Cada projeto tem uma necessidade a ser suprida, e não se atentar a nuances do processo de modelagem sistêmica pode acarretar a absorção de desvantagens com as quais o projeto não está preparado para lidar.

2.2.4 Restrições Regulatórias

Ao redor do mundo, protocolos de transmissão que utilizam ondas de rádio como meio de propagação de sinal têm seu funcionamento regulado por órgãos competentes pela homologação de dispositivos de comunicação. No Brasil, a ANATEL (Agência Nacional de Telecomunicações) é a responsável por regular, fiscalizar e homologar dispositivos com o foco de estabelecer redes de comunicação.

O AU915 é a restrição imposta pela ANATEL no Ato nº 14.448, estabelecida em 4 de dezembro de 2017 (ANATEL, 2017), em que se adota o padrão australiano de radiocomunicação. Visto que o Brasil adotou o padrão americano de distribuição de canais, o Industrial, Scientific and Medical (ISM), o AU915 foi escolhido por conta da já existente implantação de infraestrutura de rádio que ocupava parcialmente as faixas abaixo de 915 MHz; portanto, o padrão americano (US915), que funciona em uma faixa de frequência entre 902 e 928 MHz, tornou-se inadequado para implantação em território brasileiro.

O ISM foi desenvolvido em Atlanta em 1947, elaborado pela FCC (Federal Communications Commission), com o objetivo de delimitar internacionalmente os espectros de frequências de rádio para as mais diversas finalidades. Em 1985, a FCC autorizou o uso livre de espectros de rádio em bandas ISM sem a necessidade de licenciamento para utilização em determinados espectros de frequência.

Entre as regulações implementadas pelos mais diversos países ao redor do mundo, dois termos ganham grande relevância. O *duty cycle* é a relação entre o tempo em que um dispositivo ocupa um canal de rádio e o tempo total de observação. Já o *dwell time* é o tempo máximo de ocupação contínua de uma frequência por transmissão (ADAM, 2022).

Tendo o AU915 operando entre as faixas de frequência de 915 e 928 MHz, a norma brasileira não impõe restrição de duty cycle; em seu lugar, o Ato nº 14.448 estabelece que, para sistemas de salto em frequência operando nas faixas de 902–907,5 MHz e 915–928 MHz com largura de faixa de canal inferior a 250 kHz, caso do LoRa com 125 kHz, o tempo médio de ocupação de qualquer radiofrequência não deve ser superior a 0,4 segundos em um intervalo de 14 segundos, com uso mínimo de 35 canais de salto.

O mesmo Ato prevê, no item 10.3, um segundo enquadramento para equipamentos que operam nessas faixas: o de sistemas que empregam modulação digital. Nesse caso, não há exigência de salto em frequência nem limite de ocupação por canal, mas o equipamento deve atender a três requisitos: largura de faixa a 6 dB de, no mínimo, 500 kHz; potência de pico máxima de 1 W na saída do transmissor; e densidade espectral de potência de, no máximo, 8 dBm em qualquer faixa de 3 kHz. Antenas com ganho superior a 6 dBi exigem a redução da potência de transmissão (item 10.5) (ANATEL, 2017). Como os canais de 125 kHz não atingem a largura de faixa mínima, o enquadramento no item 10.3 só é possível com os canais de 500 kHz do LoRa, o que permite a operação em um canal fixo.

O AU915 utiliza 72 canais de uplink, sendo 64 de 125 kHz e 8 de 500 kHz, e 8 canais de downlink que se distribuem entre os espectros de frequência delimitados pela restrição regulatória. No caso de uplink, como mostrado pela tabela 5, os canais de 125 kHz são numerados entre 0 e 63 e distribuídos com espaçamentos de 200 kHz, iniciando em 915,2 MHz até 927,8 MHz, enquanto os de 500 kHz são numerados entre 64 e 71 e distribuídos com espaçamentos de 1,6 MHz, de 915,9 MHz até 927,1 MHz. Já no caso de downlink, como mostrado na tabela 6, os canais são numerados de 0 a 7 e distribuídos entre 923,3 e 927,5 MHz com espaçamentos de 500 kHz (LoRa Alliance Regional Parameters Workgroup, 2025).

Tabela 5 – Uplink: Canais de Frequência

Canal	**Frequência Inicial (MHz)**
0	915,2
1	915,4
2	915,6
...	...
63	927,8

Tabela 6 – Downlink: Canais de Frequência

Canal	**Frequência Inicial (MHz)**
0	923,3
1	923,8
2	924,3
...	...
7	927,5

O termo salto de frequência especificado pelo AU915 trata-se de um padrão de envio de pacotes em canais de frequência. Na tecnologia LoRa, adotam-se dois padrões de envio: o Chirp Spread Spectrum (CSS), no qual o uplink é transmitido em um canal escolhido de forma pseudoaleatória dentro dos limites regulatórios especificados, em que cada pacote é enviado de forma atomizada, e, em downlink, a mensagem é enviada no mesmo canal em que o uplink foi recebido (Semtech Corporation, 2024a); e o Long Range Frequency Hopping Spread Spectrum (LR-FHSS), que tem por objetivo ampliar a capacidade de escalonamento do sistema, sendo aplicável apenas em uplink, funcionando de forma a segmentar a mensagem em diversos pacotes e distribuí-los em diversos canais, assim prevenindo a sobrecarga de canal (LoRa Alliance Regional Parameters Workgroup, 2025).

Outro importante ponto para adequação aos limites regulatórios brasileiros é a configuração de Data Rate (DR) e, por consequência, do SF. Como explicado pela LoRa Alliance Regional Parameters Workgroup (2025), DR é um indicador de velocidade de transmissão de dados em pacote, assim como o SF, organizado em classes que vão de 0 a 15, em que, quanto menor a classe, menor é a taxa de transmissão de dados em bit por requisição. Este indicador está extremamente vinculado ao SF, de forma que quanto menor a classe de SF, maior é a classe de DR, evidenciando, assim, menor tempo de pacote no ar, e, ao contrário, quanto maior a classe SF, menor é a classe DR. As tabelas 7 e 8 demonstram como cada classe de DR impacta no funcionamento de um sistema baseado em tecnologia LoRa.

Tabela 7 – Taxas de transmissão Uplink e configurações LoRaWAN.

Uplink Data Rate	**Configuration**	**Indicative Physical Bit Rate [bit/sec]**
0	LoRa: SF12 / 125 kHz	250
1	LoRa: SF11 / 125 kHz	440
2	LoRa: SF10 / 125 kHz	980
3	LoRa: SF9 / 125 kHz	1760
4	LoRa: SF8 / 125 kHz	3125
5	LoRa: SF7 / 125 kHz	5470
6	LoRa: SF8 / 500 kHz	12500
7	LR-FHSS CR1/3: 1.523 MHz BW	162
8	RFU	
9	LoRa: SF6 / 125 kHz	9375
10	LoRa: SF5 / 125 kHz	15625
11-14	RFU	
15	Defined in [TS001][3]	

Fonte: Adaptado de LoRa Alliance, (LoRa Alliance Regional Parameters Workgroup, 2025).

Tabela 8 – Parâmetros de Downlink Data Rate

Downlink Data Rate	**Configuration**	**Indicative Physical Bit Rate [bit/sec]**
0	LoRa: SF5 / 500 kHz	62500
1-7	RFU	
8	LoRa: SF12 / 500 kHz	980
9	LoRa: SF11 / 500 kHz	1760
10	LoRa: SF10 / 500 kHz	3900
11	LoRa: SF9 / 500 kHz	7000
12	LoRa: SF8 / 500 kHz	12500
13	LoRa: SF7 / 500 kHz	21900
14	LoRa: SF6 / 500 kHz	37500
15	Defined in [TS001][3]	

Fonte: Adaptado de LoRa Alliance, (LoRa Alliance Regional Parameters Workgroup, 2025).

O DR impacta também na configuração de recebimento de mensagens. Como explicado anteriormente, quando o dispositivo final estiver em classe A e enviar uma requisição uplink, o servidor de rede, ao receber a mensagem, devolverá uma requisição downlink.

Essa última requisição será lida em uma janela de tempo específica, e sob um data rate especificado em uma tabela. A tabela 9 especifica as possíveis configurações da janela de recebimento (chamada de RX1), que entram em conformidade com o AU915.

Tabela 9 – Downlink RX1 Data Rate Mapping

Uplink Data Rate	**Downlink Data Rate**					
`RX1DROffset`	**0**	**1**	**2**	**3**	**4**	**5**
DR0	DR8	DR8	DR8	DR8	DR8	DR8
DR1	DR9	DR8	DR8	DR8	DR8	DR8
DR2	DR10	DR9	DR8	DR8	DR8	DR8
DR3	DR11	DR10	DR9	DR8	DR8	DR8
DR4	DR12	DR11	DR10	DR9	DR8	DR8
DR5	DR13	DR12	DR11	DR10	DR9	DR8
DR6	DR13	DR13	DR12	DR11	DR10	DR9
DR7	DR9	DR8	DR8	DR8	DR8	DR8
DR9	DR14	DR13	DR12	DR11	DR10	DR9
DR10	DR0	DR14	DR13	DR12	DR11	DR10

Fonte: Adaptado de LoRa Alliance, (LoRa Alliance Regional Parameters Workgroup, 2025).

Outro fator importante a determinar dentro de um sistema LoRa é o Effective Isotropic Radiated Power (EIRP), que é o valor em dBm que representa a potência do sinal de rádio emitido pelo hardware. O valor máximo permitido pela regulação AU915 é de +30 dBm. O EIRP é determinado pela fórmula 2.1, em que o parâmetro *TXPower* é configurável via software no dispositivo final.

$$\text{EIRP} = \text{MAX EIRP} - 2 \times \text{TXPower} \tag{2.1}$$

2.2.5 Inteligência Artificial

2.2.5.1 Machine Learning

Segundo Vilela (2021), Machine Learning pode ser definida como um ramo da Inteligência Artificial (IA) responsável por capacitar computadores a simular aprendizado com base em processamento de dados. Essa técnica está em ascensão atualmente, sendo amplamente aplicada em softwares que têm por objetivo a identificação de padrões, como é o caso do OCR, detecção de phishing em e-mails, detecção de uso fraudulento de cartões de crédito, entre outras aplicações.

Tal subconjunto tem por característica a distinção de métodos de aprendizagem, tendo como os mais difundidos os métodos supervisionado, não supervisionado e semissupervisionado.

2.2.5.1.1 Aprendizado Supervisionado

Neste método de aprendizagem, o objetivo é ensinar ao modelo de IA, através de objetos previamente rotulados, a identificar características de modo a classificar novos objetos (VILELA, 2021). Entre os algoritmos mais importantes para a aplicação deste tipo de método de treinamento estão:

1. k-Nearest Neighbours;
2. Regressão Linear;
3. Máquina de Vetores de Suporte (SVM);
4. Árvore de Decisão e Floresta Aleatória;
5. Redes Neurais;

Observação: estes algoritmos não necessariamente são concorrentes; portanto, em um único modelo, pode haver implementação de um ou mais dos algoritmos citados (VILELA, 2021).

2.2.5.1.2 Deep Learning

Deep Learning, ou aprendizado profundo, é uma técnica que se encontra em um subcampo do Machine Learning, cujo desenvolvimento foi pautado na simulação do processo de aprendizagem humana. Nesse caso, utiliza-se como base o algoritmo de redes neurais, construindo estruturas conectadas de forma a emular o comportamento do cérebro humano (JESUS; COQUEIRO, 2023).

Para o processamento de dados sequenciais ou cronológicos, como linhas de texto em sistemas de reconhecimento de caracteres, as arquiteturas profundas evoluem para estruturas capazes de reter dependências temporais.

2.2.5.1.3 O que são LSTMs.

A *Long Short-Term Memory* (LSTM) é uma arquitetura de rede neural recorrente proposta por Hochreiter e Schmidhuber (1997) para superar a limitação das RNNs tradicionais em aprender dependências de longo prazo, problema causado pelo desaparecimento do gradiente durante a retropropagação (HOCHREITER; SCHMIDHUBER, 1997). A unidade básica da LSTM é a célula de memória, cujo estado interno é controlado por

três portas multiplicativas: a **porta de esquecimento** (*forget gate*), que decide quais informações do estado anterior devem ser descartadas; a **porta de entrada** (*input gate*), que seleciona quais novas informações serão armazenadas no estado da célula; e a **porta de saída** (*output gate*), que define o que será exposto como saída da célula naquele passo de tempo (HOCHREITER; SCHMIDHUBER, 1997; HANNUN, 2017). Esse mecanismo de portas permite que a LSTM mantenha informações relevantes por muitos passos de tempo e descarte informações irrelevantes, o que é essencial para o reconhecimento de sequências textuais, nas quais o contexto de caracteres anteriores e posteriores influencia a interpretação de cada símbolo (SHI; BAI; YAO, 2017).

2.2.5.1.4 Limitações em *Edge AI* e alternativas convolucionais.

Apesar de sua eficácia na modelagem de contexto, as LSTMs impõem desafios significativos em dispositivos de borda com restrições severas de memória e poder computacional. A natureza sequencial da recorrência impede o paralelismo ao longo do eixo temporal, e o estado interno da célula precisa ser mantido entre passos de tempo, o que eleva o consumo de memória e a latência de inferência em microcontroladores (NGUYEN et al., 2026). Além disso, a implementação eficiente de operadores recorrentes em *runtimes* C++ para *edge* ainda é limitada, com suporte incompleto em algumas bibliotecas de inferência para microcontroladores (NGUYEN et al., 2026). Como alternativa, empregam-se **convoluções temporais 1D com núcleos largos** (ou convoluções dilatadas), que emulam o campo receptivo contextual da recorrência sem o custo computacional associado ao estado recorrente, permitindo processamento paralelo e maior eficiência em hardware embarcado (NAZRE et al., 2024; SHI; BAI; YAO, 2017).

2.2.5.2 Redes Neurais

Como apresentado por Jesus e Coqueiro (2023), redes neurais são estruturas baseadas em grafos, representadas em camadas; cada camada é composta por nós, denominados neurônios, e arestas, denominadas sinapses. A estrutura básica de uma rede neural possui três camadas: entrada, camada intermediária e saída, que são, respectivamente, responsáveis por executar as funções de receber os dados externos, processar os dados e entregar o resultado.

A técnica de redes neurais apresenta derivações amplamente utilizadas como alicerce no desenvolvimento de sistemas baseados em Deep Learning.

2.2.5.2.1 Redes Neurais Convolucionais

Atualmente, as redes neurais convolucionais são amplamente aplicadas em processamento de dados não estruturados, como imagens, vídeos e áudios. Tal técnica tem como característica o processamento de pequenos trechos de dados por vez, ao contrário de outras

técnicas que tentam realizar o processamento de forma atômica (JESUS; COQUEIRO, 2023).

Como apresentado por Atika et al. (2022), o desenvolvimento de uma Convolutional Neural Network (CNN) inicia-se pela definição da arquitetura em implementação. Tal técnica abrange uma vasta quantidade de arquiteturas que foram desenvolvidas para auxiliar e alcançar melhores resultados para determinados requisitos do software.

2.2.5.2.2 Camadas que Constituem uma CNN

Uma CNN é constituída por três principais camadas: camada de entrada de dados, camada de saída, também denominada camada densa, na qual cada um de seus neurônios está conectado à camada anterior, e diversas outras camadas intermediárias, denominadas por alguns autores camadas ocultas (AYENI, 2022).

As camadas intermediárias de uma CNN apresentam comportamento de aprofundamento em reconhecimento de abstrações a cada camada sobreposta, de tal forma que as camadas iniciais são responsáveis por identificar traços, contornos, linhas etc., enquanto as camadas finais possuem a responsabilidade de identificar objetos, rostos e valores, por exemplo. Tais camadas são desenvolvidas com a intercalação de camadas de convolução e pooling (agrupamento), de forma que cada entrada de uma camada é a saída da camada anterior.

2.2.5.2.3 Camada Convolucional

Como Ayeni (2022) explica, a camada convolucional é a camada responsável por processar imagens de forma a encontrar padrões ao aplicar um filtro (kernel/neurônio) a cada trecho da imagem de entrada, denominado campo receptivo. O neurônio é responsável por aplicar operações de multiplicação sobre o campo receptivo, gerando um único valor para cada trecho de imagem processado, e, posteriormente, os valores calculados pelos neurônios são armazenados em uma estrutura de três dimensões (AYENI, 2022).

A conversão da imagem para escala de cinza antes do processamento por uma CNN é justificada por reduzir a complexidade cromática do padrão Red, Green, Blue (RGB) (três canais) para um canal único, o que impacta diretamente a profundidade dos kernels e o volume de parâmetros da primeira camada da rede (AYENI, 2022).

2.2.5.2.4 Camada de Pooling

Segundo Ayeni (2022), a camada de pooling possui a responsabilidade de filtrar características predominantes encontradas na camada de convolução. Tal camada possui a importantíssima função de otimizar processamento, descartando dados menos significativos e mitigando o fenômeno de overfitting. As camadas de pooling possuem diversas derivações,

que são escolhidas e implementadas a fim de otimizar resultados alcançados pelo modelo, mas a derivação mais aplicada sobre CNNs é a Max Pooling.

A operação de pooling reduz a resolução espacial dos mapas de características gerados pela camada convolucional, utilizando tipicamente uma janela de varredura (filtro) de 2×2.

2.2.5.2.5 Camada Totalmente Conectada

Como Ayeni (2022) explica, a camada denominada Fully-Connected (Totalmente Conectada) possui a função de interpretar abstrações mais complexas da imagem e classificá-las. Essa camada possui todos os seus parâmetros conectados com os parâmetros da camada anterior, corriqueiramente resultando em um número de nós de saída igual à quantidade de classes de classificação.

2.2.5.3 OCR - Fundamentos e Escopo

O OCR é um subcampo da Visão Computacional e da Inteligência Artificial dedicado à conversão de imagens contendo texto (impresso, manuscrito ou gravado) em dados legíveis por máquina (CHEN et al., 2021). Originalmente voltado à digitalização de documentos, o escopo foi ampliado pelo *Deep Learning* para aplicações industriais e de IoT (JAWAS; INDRIANTO, 2018).

2.2.5.3.1 Pipeline Tradicional

Um sistema clássico de OCR opera em etapas interdependentes (CHEN et al., 2021):

1. **Aquisição e pré-processamento**: melhoria de contraste, correção de inclinação (*deskewing*), remoção de ruído e binarização; em dispositivos embarcados, etapas de segmentação semântica podem isolar a Região de Interesse (ROI) antes do reconhecimento.

2. **Localização e segmentação de texto**: identificação das regiões de texto.

3. **Reconhecimento de caracteres**: classificação dos segmentos ou da sequência completa.

4. **Pós-processamento**: dicionários, correção ortográfica ou verificação sintática para mitigar erros residuais.

Em OCR, a leitura de uma palavra ou sequência numérica se beneficia do contexto em ambas as direções: o caractere atual é mais bem interpretado quando se conhece tanto o que veio antes quanto o que veio depois. Por isso, a CRNN emprega tipicamente camadas **bidirecionais** de LSTM (BiLSTM), que processam a sequência da esquerda para a direita

e da direita para a esquerda e concatenam as representações resultantes (SHI; BAI; YAO, 2017; HANNUN, 2017).

2.2.5.3.2 Evolução Tecnológica: dos Métodos Heurísticos ao Deep Learning

A literatura descreve três gerações principais (CHEN et al., 2021):

Primeira geração (Template Matching). Sobreposição pixel a pixel com gabaritos estáticos; alta vulnerabilidade a variações de escala, rotação e fontes.

Segunda geração (Características manuais + classificadores clássicos). Uso de descritores como HOG e SIFT, combinados com SVM ou MLP; dependência de *feature engineering*.

Terceira geração (End-to-End Deep Learning). Elimina a segmentação explícita de caracteres; modelos *end-to-end* aprendem simultaneamente representações visuais e dependência contextual sequencial (SHI; BAI; YAO, 2017).

2.2.5.4 Arquitetura U-Net - Segmentação de ROI

Dado o objetivo da implementação de CNN neste trabalho, destaca-se a arquitetura U-Net.

Como Atika et al. (2022) explicam, a arquitetura U-Net tem por objetivo a definição de bordas dentro de imagens, sendo amplamente utilizada na biomedicina com o foco de delimitar estruturas presentes em imagens; tal característica é explorada a fim de definir Region of Interest (ROI) dentro de uma imagem.

A arquitetura U-Net é caracterizada pela presença de três etapas de processamento: Contração, Gargalo e Expansão. A camada de contração é composta por vários blocos, nos quais cada bloco recebe uma entrada e aplica uma camada de convolução seguida por uma camada de operação de max-pooling; essas camadas são matrizes que, respectivamente, apresentam a estrutura 3x3 e 2x2.

Com adendo, as camadas que constroem a arquitetura U-Net utilizam a função de ativação ReLU apresentada na equação 2.2.

$$f(x) = \text{ReLU}(x) = \max(0, x) = \begin{cases} x, & \text{se } x > 0 \\ 0, & \text{se } x \leq 0 \end{cases} \tag{2.2}$$

2.2.5.5 Arquitetura CRNN e CTC

A arquitetura Convolutional Recurrent Neural Network (CRNN) é amplamente utilizada em aplicações que exigem inferência de classes em sequência, pois combina um extrator

convolucional (CNN) a fim de obter representações visuais, uma camada recorrente (Recurrent Neural Network (RNN)), com o objetivo de modelar dependências contextuais ao longo do eixo horizontal, e a função de perda Connectionist Temporal Classification (CTC), com a finalidade de alinhar as previsões da rede à sequência de rótulos sem a necessidade de segmentação explícita dos caracteres (SHI; BAI; YAO, 2017).

A técnica CTC resolve o problema central de sequências de comprimento variável: em geral, não se sabe onde cada caractere começa e termina na imagem, e o número de *frames* produzidos pela CNN é maior do que o número de caracteres da palavra ou número a ser reconhecido (GRAVES et al., 2006). Para lidar com isso, a técnica CTC introduz dois mecanismos fundamentais: um *token* oculto de espaçamento (*blank*) e uma função de colapso que mapeia a sequência de previsões da rede para o rótulo final (GRAVES et al., 2006; HANNUN, 2017).

O *token blank* e o colapso de repetições. A cada passo de tempo, a rede produz uma distribuição de probabilidade sobre o alfabeto acrescido do símbolo especial *blank* (ϵ). O *blank* representa a ausência de emissão de caractere naquele instante, funcionando como um separador entre caracteres idênticos adjacentes (GRAVES et al., 2006). Após a rede produzir a sequência completa de previsões, aplica-se uma função de colapso $\mathcal{B}$ que (i) remove caracteres repetidos consecutivos e (ii) remove os *blanks* (HANNUN, 2017). Por exemplo, a sequência de caminho $--h--e--l--l--o--$ colapsa para "hello"; sem o *blank*, as repetições de "l" seriam indistinguíveis de um único "l" (HANNUN, 2017).

Marginalização de alinhamentos. O treinamento da CTC não exige um alinhamento predefinido entre os *frames* e os caracteres. Em vez disso, a função de perda calcula a probabilidade da sequência de rótulos correta como a soma das probabilidades de todos os caminhos de alinhamento válidos que, após o colapso, produzem aquele rótulo (GRAVES et al., 2006). Essa marginalização é feita de forma eficiente pelo algoritmo *forward-backward*, que explora a estrutura do problema para evitar a enumeração explícita de todos os caminhos, que seria computacionalmente inviável (GRAVES et al., 2006; HANNUN, 2017). Assim, a rede pode ser treinada diretamente com pares (imagem, transcrição), sem anotações de posição caractere a caractere.

Inferência. No momento da inferência, a rede produz, para cada passo de tempo, a classe mais provável (ou uma distribuição sobre as classes). A sequência final é obtida pelo colapso das previsões, opcionalmente combinado com busca em feixe (*beam search*) para explorar múltiplos alinhamentos e melhorar a acurácia (HANNUN, 2017). O artigo de Hannun (2017) apresenta uma explicação visual detalhada desse processo, incluindo exemplos passo a passo de como o *blank* e o colapso atuam em sequências reais (HANNUN, 2017).

Por que a CTC é adequada ao OCR. Em OCR de palavras, números de medidores ou placas, o número de caracteres varia e não há garantia de que cada caractere ocupe exatamente um número fixo de *frames* na saída da CNN. A CTC elimina a necessidade de segmentação manual e permite que a CRNN seja treinada de ponta a ponta com sequências de comprimentos diferentes, mantendo a coerência entre as previsões vizinhas e evitando o colapso de repetições idênticas (SHI; BAI; YAO, 2017; GRAVES et al., 2006).

2.2.5.6 Quantização e QAT

Quantização é um método de simplificação dos parâmetros de um modelo de Redes Neurais Artificiais (RNA). Como Ferreira (2022) explica, a aplicação de tal técnica traz como benefício a redução do tamanho de modelos, viabilizando a utilização em equipamentos de baixo poder computacional, além do aumento de velocidade de inferência e simplificação de cálculos realizados pelo processador, frente a um modelo que, na maioria dos casos, possui seus parâmetros representados em pontos flutuantes de 32 bits.

A técnica Quantization-Aware Training (QAT), com foco em transformação de parâmetros para inteiros de 8 bits, simula a perda de precisão da conversão de FP32 para INT8 durante o treinamento, permitindo que os pesos da rede se adaptem e mitiguem a degradação de acurácia. Em sistemas de OCR para medidores, modelos quantizados alcançam tamanhos da ordem de centenas de kB e inferência em subsegundo em hardware embarcado (ADEMOLA; PETLENKOV; LEIER, 2023; JAWAS; INDRIANTO, 2018).

2.3 Trabalhos correlatos

Nesta seção, analisam-se pesquisas que utilizam IoT e Visão Computacional para a digitalização de medidores de consumo, como as soluções de computação de borda propostas por Vilela (2021) e Bravo, Javier e Tolentino (2025), além de arquiteturas de sensores inteligentes exploradas por Tedjojuwono e Jahja (2024). O objetivo é contextualizar as escolhas arquiteturais deste TCC, comparando modelos de processamento (borda vs nuvem) e protocolos de comunicação sem fio. A revisão fundamenta o estado da técnica e destaca os diferenciais desta proposta em termos de infraestrutura de dados e eficiência energética.

2.3.1 Trabalho 1: Solução IoT para Digitalização de Medidores de Consumo por Visão Computacional e MicroML

Vilela (2021) abordou o tema de soluções IoT para Medidores de Consumo. Para resolvê-lo, foi desenvolvido um sistema utilizando microprocessadores ESP32, baseado no método de Edge-Computing (Computação em Borda). O autor desenvolveu um software próprio, otimizado para microprocessadores, focado em OCR, extraindo os dados, transmitindo-os

via rede LoRaWan e armazenando-os localmente; posteriormente, os dados são apresentados em um simples dashboard apresentando o consumo extraído pelo sistema até o momento.

A metodologia utilizada foi a pesquisa experimental, na qual houve o desenvolvimento de um sistema que supre a necessidade de extração de dados de maneira digital.

Os resultados mostraram que o desenvolvimento deste tipo de ferramenta é totalmente viável e, em suma, o experimento foi bem-sucedido. No entanto, a solução apresenta limitações, principalmente no que tange ao desenvolvimento do dataset, no qual não foi implementado tratamento adequado ao enviesamento dos dados. Também existe uma carência de métricas para medição de consumo energético e de processamento pelo sistema.

O presente trabalho diferencia-se por aplicar uma comparação entre uma arquitetura clássica de Edge vs Cloud, e um ponto que atacamos mais profundamente é a apresentação de métricas importantíssimas no desenvolvimento de um sistema IoT além da acurácia de modelo, por exemplo, as métricas de consumo energético e impactos em manutenção por decisões tomadas.

2.3.2 Trabalho 2: Sistema de Gerenciamento Inteligente de Água Utilizando Sensores Baseados em IoT

Tedjojuwono e Jahja (2024) abordaram o tema de sistemas inteligentes para o manejo de recursos hídricos em áreas residenciais. Para resolvê-lo, os autores propuseram o desenvolvimento de um framework que substitui medidores mecânicos por sensores de fluxo eletrônicos integrados a uma arquitetura IoT. O sistema utiliza prototipagem com Arduino Uno e sensores de efeito Hall (YF-S201) para coletar dados de vazão em tempo real, enviando as informações via módulo 3G/GPRS para um servidor de banco de dados centralizado. Os dados processados alimentam uma aplicação web que permite o monitoramento do consumo, faturamento automatizado e a detecção precoce de anomalias na distribuição.

A metodologia utilizada seguiu os princípios do Design Thinking, incluindo uma fase qualitativa de coleta de requisitos através de entrevistas com usuários e a criação de mapas de empatia, seguida pelo desenvolvimento do protótipo e avaliação de usabilidade através da escala SUS (System Usability Scale).

Os resultados mostraram que a solução é viável e intuitiva, obtendo uma pontuação média de 72 na escala SUS, o que indica uma boa aceitação pelos usuários. No entanto, o trabalho aponta limitações na abrangência dos dados, focando primariamente na taxa de fluxo, e destaca a necessidade de integrar mais tipos de sensores (como pressão e qualidade da água) para análises de Big Data mais robustas. Além disso, a precisão total do sistema de previsão depende de um volume maior de dados históricos que ainda precisam ser coletados.

O presente trabalho diferencia-se por focar na infraestrutura de comunicação LoRaWan,

visando uma maior eficiência energética e alcance em comparação ao GPRS/3G utilizado pelos autores, além de explorar a integração direta com medidores analógicos já existentes através de telemetria, em vez da substituição completa do hardware de medição.

2.3.3 Trabalho 3: Rede de Sensores Sem Fio Habilitada para LoRa para Leitura e Faturamento de Consumo de Água Usando Reconhecimento Óptico de Caracteres

Bravo, Javier e Tolentino (2025) desenvolveram um sistema automatizado para leitura de hidrômetros analógicos utilizando uma combinação de tecnologias de baixo custo e comunicação de longo alcance. O objetivo central foi mitigar erros humanos e atrasos operacionais comuns em leituras manuais.

A metodologia consistiu na integração de um módulo ESP32-CAM para captura de imagens dos medidores e um algoritmo de OCR para a extração digital dos dados de consumo no próprio dispositivo. Para a transmissão, utilizou-se a tecnologia LoRa (módulo Ra-02), operando em uma topologia de rede que inclui repetidores para estender o alcance e um gateway central para envio dos dados à nuvem. O sistema também implementou estratégias de gerenciamento de energia através de ciclos de *deep sleep* programados para aumentar a vida útil da bateria.

Os resultados apontaram uma precisão de 99,71% no reconhecimento dos dígitos e um alcance de transmissão estável de até 1070 metros em condições de campo aberto. Os autores observaram que medidores mais antigos ou obstruídos podem elevar o tempo de processamento do OCR.

O presente trabalho diferencia-se da abordagem de Bravo, Javier e Tolentino (2025) ao deslocar a carga computacional do OCR do dispositivo final para a nuvem. Enquanto o trabalho correlato executa o processamento localmente no ESP32-CAM, esta pesquisa foca na robustez de um pipeline de processamento remoto. Isso permite a utilização de modelos de visão computacional mais complexos e precisos, que excederiam a capacidade de memória e processamento do hardware local, além de centralizar a lógica de extração de dados para facilitar manutenções e atualizações do algoritmo de IA sem a necessidade de novos firmwares nos sensores. Esta abordagem também visa mitigar as dificuldades de processamento em medidores antigos ou obstruídos, um desafio observado na implementação local de Bravo, Javier e Tolentino (2025).

Utilizaremos esse trabalho como referência de borda para realizar uma comparação com a abordagem de processamento em nuvem, contribuindo para a discussão sobre o trade-off entre eficiência energética e precisão em sistemas de telemetria de baixo custo.

3 Desenvolvimento / Implementação / Experimentos

3.1 Metodologia da pesquisa

Para avaliar a eficácia do sistema de telemetria proposto e garantir a objetividade dos resultados, serão adotados indicadores quantitativos divididos em três dimensões críticas: desempenho de rede, precisão de visão computacional, eficiência energética e acurácia preditiva.

3.1.1 Desempenho da Rede de Comunicação

A robustez da transmissão via LoRaWan será quantificada através da Packet Delivery Ratio (PDR). Conforme a metodologia de Bravo, Javier e Tolentino (2025), o PDR é definido pela razão entre o número de pacotes recebidos pelo *gateway* e o número de pacotes enviados pelo nó sensor:

$$PDR = \left(\frac{\text{Pacotes Recebidos}}{\text{Pacotes Enviados}} \right) \times 100 \tag{3.1}$$

Além do PDR, será medida a latência de ponta a ponta, compreendendo o tempo decorrido desde o disparo da captura da imagem no ESP32 até a disponibilidade do dado processado no *dashboard*.

3.1.2 Acurácia da Visão Computacional (OCR)

A validação do algoritmo de OCR, responsável pela extração dos dígitos dos medidores, será realizada sob diferentes condições de iluminação e ângulos de captura, visando simular cenários reais de instalação.

Conforme discutido por Barkat (2024), a eficácia de modelos de *Deep Learning* aplicados ao monitoramento deve ser medida pela sua capacidade de generalização. Para este trabalho, a métrica principal será a Acurácia de Leitura (Acc_{ocr}), definida pela razão entre as predições que coincidem exatamente com o valor real (verdade de campo) e o total de amostras testadas:

$$Acc_{ocr} = \frac{VP + VN}{VP + VN + FP + FN} \tag{3.2}$$

Em que VP, VN, FP e FN representam, respectivamente, os Verdadeiros Positivos, Verdadeiros Negativos, Falsos Positivos e Falsos Negativos obtidos através da Matriz de Confusão. Esta abordagem quantitativa permite identificar se erros de leitura estão

associados a fatores externos, como reflexos na cúpula de vidro do medidor ou distorções de perspectiva.

3.1.3 Autonomia Energética

A eficiência do hardware será validada através da medição do consumo de corrente em diferentes estados de operação (*Deep Sleep*, captura de imagem e transmissão). A autonomia estimada (A) em dias será calculada pela relação entre a capacidade da bateria (C_{bat}) e o consumo médio diário ($I_{médio}$):

$$A = \frac{C_{bat}}{I_{médio} \times 24} \tag{3.3}$$

3.2 Detalhes da implementação

3.2.1 Visão geral da solução

O sistema desenvolvido realiza a leitura automática de hidrômetros analógicos do tipo ciclométrico, cujo mostrador é formado por roletes numerados, por meio de um nó sensor baseado no ESP32-CAM. O nó fotografa o mostrador do medidor e envia o resultado, via LoRaWan, até uma estação receptora conectada a um computador, que faz o papel de servidor da aplicação.

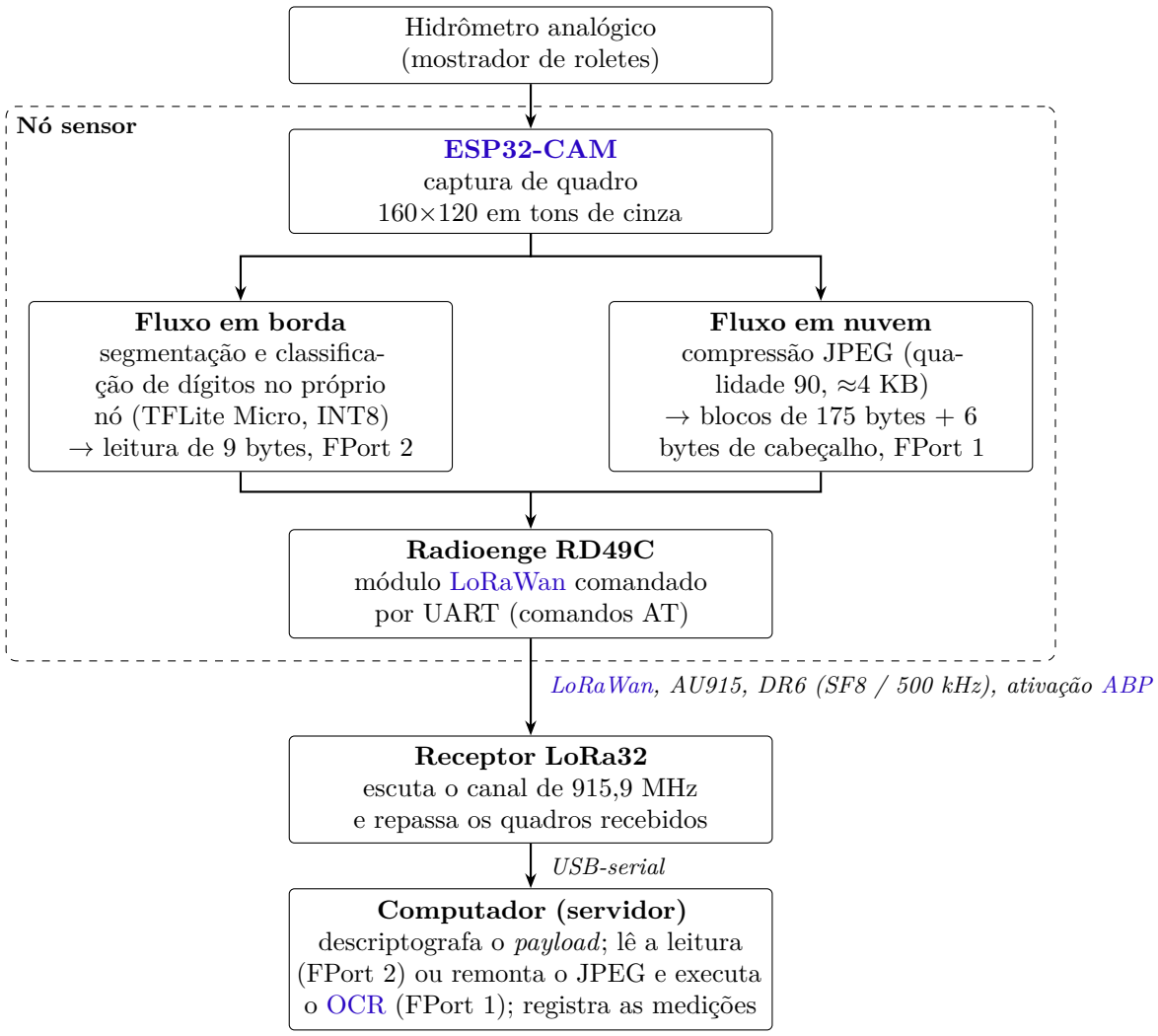
Para responder à pergunta de pesquisa, o mesmo nó foi implementado em dois fluxos de processamento, que diferem no local em que o OCR é executado e, por consequência, no conteúdo que trafega pelo enlace:

- **Fluxo em nuvem** (Cloud Computing): o nó captura a imagem, comprime-a no formato JPEG e a transmite fragmentada em blocos. O servidor remonta a imagem e executa o reconhecimento dos dígitos.
- **Fluxo em borda** (Edge Computing): o nó captura a imagem, executa localmente os modelos de reconhecimento e transmite apenas a leitura resultante, em um único pacote de 9 bytes.

Uma decisão central da implementação é que os dois fluxos utilizam os mesmos modelos de redes neurais, quantizados em inteiros de 8 bits (INT8), e a mesma lógica de leitura. Assim, as diferenças de latência, entrega de pacotes e consumo energético observadas entre eles decorrem apenas do local de processamento e do volume de dados transmitido, e não de modelos com capacidades distintas. O efeito da quantização sobre a acurácia, objeto da hipótese H1, é avaliado à parte, pela comparação entre as versões em ponto flutuante de 32 bits e em INT8 do mesmo modelo.

A Figura 7 apresenta a arquitetura do sistema e o caminho percorrido pelos dados em cada fluxo.

Figura 7 – Arquitetura do sistema e os dois fluxos de processamento



Fonte: Elaborado pelos autores (2026).

3.2.2 Componentes de hardware

O protótipo foi montado com componentes de baixo custo. A Tabela 10 resume a função de cada um.

Tabela 10 – Componentes de hardware do protótipo

Componente	**Função no sistema**
ESP32-CAM (AI-Thinker)	Nó sensor. Microcontrolador ESP32 a 240 MHz com 4 MB de PSRAM e câmera integrada. Captura a imagem e, no fluxo em borda, executa os modelos de OCR.
Radioenge RD49C	Módulo LoRaWan homologado pela ANATEL, ligado ao ESP32-CAM por UART (GPIO 15 e GPIO 14) e controlado por comandos AT. Opera no plano AU915 com potência de até +20 dBm.
Heltec/TTGO LoRa32 V2	Receptor. Rádio LoRa configurado no mesmo canal e na mesma taxa do transmissor; repassa cada quadro recebido ao computador pela porta serial, junto com o RSSI.
Computador	Servidor da aplicação. Descriptografa os quadros, remonta as imagens, executa o OCR do fluxo em nuvem e registra as medições. Também foi usado no treinamento dos modelos.
INA219 e Arduino Uno	Instrumentação. Sensor de corrente com resistor *shunt* de 0,1 Ω, lido por um Arduino Uno a 100 amostras por segundo, usado para medir o consumo do nó sensor.

Fonte: Elaborado pelos autores (2026).

3.2.3 Componentes de software

O código do projeto está reunido em um único repositório[1] e se divide em firmwares para os microcontroladores, escritos em C++ com o *core* Arduino-ESP32 3.0.7, e *scripts* em Python executados no computador. A Tabela 11 relaciona os principais componentes.

[1] Disponível em <https://github.com/Marcelo-122/Codigo-do-TCC2>.

Tabela 11 – Principais componentes de software

Componente	**Responsabilidade**
`transmissor_esp32cam`	Firmware do nó no fluxo em nuvem: captura, compressão JPEG, fragmentação e envio dos blocos pela porta 1 (FPort 1).
`transmissor_ocr_esp32cam`	Firmware do nó no fluxo em borda: captura, inferência dos modelos com TensorFlow Lite Micro e envio da leitura pela porta 2 (FPort 2).
`receptor_lora32`	Firmware do receptor: escuta o canal e escreve na serial cada quadro recebido.
`reconstrutor.py`, `enlace.py`	Lado do servidor: leitura da serial, descriptografia, remontagem da imagem ou decodificação da leitura e registro das medições.
`ocr/pipeline_nuvem.py`	OCR do fluxo em nuvem, com os mesmos modelos e a mesma lógica do firmware de borda.
`ModeloEDataset/`	Preparação dos conjuntos de dados, treinamento, quantização e exportação dos modelos para o firmware.
`medicoes/`	*Scripts* de medição de corrente e de latência e os dados brutos coletados.

Fonte: Elaborado pelos autores (2026).

Nos dois fluxos, os firmwares do nó compartilham o mesmo rádio, a mesma configuração de enlace, o mesmo ciclo de *deep sleep* e o mesmo formato de registro de tempos, de modo que as medições de consumo e de latência sejam obtidas pelos mesmos *scripts* e possam ser comparadas diretamente.

3.2.4 Simplificações em relação a uma implantação real

Algumas escolhas do protótipo simplificam a infraestrutura descrita na Seção 2.2 e devem ser consideradas na interpretação dos resultados:

- **Receptor de canal único no lugar de um *gateway***: o receptor LoRa32 escuta um único canal, fixo, e não implementa as funções de um *gateway* LoRaWan multicanal. Por isso, o transmissor também foi configurado para usar um único canal (Seção 3.2.5).
- **Ausência de servidor de rede**: não há servidor de rede nem servidor de aplicação dedicados. O dispositivo usa ativação ABP, com chaves de sessão configuradas previamente, e o próprio computador descriptografa o *payload* com a chave de aplicação (AppSKey). O código de integridade da mensagem (MIC) não é verificado, pois exigiria a chave de rede. Um protótipo inicial, que injetava os pacotes em um servidor ChirpStack sem uso de rádio, foi descartado.

- **Ambiente de bancada**: as medições de enlace foram feitas com transmissor e receptor a poucos metros de distância, com sinal forte, de modo que avaliam o comportamento dos fluxos, e não o alcance do enlace.

3.2.5 Enlace LoRaWAN

Esta seção descreve a configuração do enlace de rádio, o formato das mensagens trocadas nos dois fluxos, o preparo e a fragmentação da imagem e o tratamento dos quadros recebidos pelo servidor. Os mesmos parâmetros de rádio foram usados nos dois fluxos.

3.2.5.1 Estudo preliminar e escolha da configuração de rádio

Na fase inicial do trabalho, a viabilidade do envio de imagens foi estimada analiticamente para o plano AU915 com canais de 125 kHz e SF 8. Nessa configuração, o Ato nº 14.448 da ANATEL enquadra o LoRa como sistema de salto em frequência, que deve usar no mínimo 35 canais e limitar o tempo médio de ocupação de cada canal a 0,4 s em um intervalo de 14 s (ANATEL, 2017). Para respeitar esse limite, cada pacote foi restrito a 115 bytes na camada física; com isso, uma imagem JPEG de 3.182 bytes exigiria 38 pacotes de 348,7 ms cada, totalizando cerca de 13,3 s de rádio ativo.

Essa configuração foi abandonada na implementação por dois motivos. Primeiro, o salto entre pelo menos 35 canais é incompatível com o receptor disponível, que escuta um único canal (Seção 3.2.4). Segundo, o limite de 400 ms por transmissão restringe o tamanho do bloco de dados: um pacote com 181 bytes de *payload*, tamanho adotado adiante, ocuparia 543 ms de rádio com 125 kHz.

Por isso, o enlace foi configurado para operar com largura de banda de 500 kHz, no *data rate* DR6 do plano AU915, que corresponde a SF8 com 500 kHz (Tabela 7) e a um *payload* máximo de 222 bytes (Radioenge, 2022). Com essa largura de banda, o rádio se enquadra no item 10.3 do mesmo Ato, destinado a sistemas de modulação digital (Seção 2.2.4), cujos três requisitos são verificados a seguir: largura de faixa a 6 dB de, no mínimo, 500 kHz; potência de pico máxima de 1 W; e densidade espectral de potência máxima de 8 dBm em qualquer faixa de 3 kHz.

O módulo Radioenge RD49C transmite com potência máxima de +20 dBm (100 mW), sem comando AT que permita ajustá-la (Radioenge, 2022), abaixo do limite de 1 W. Supondo que essa potência se distribua de forma plana pelos 500 kHz, a densidade espectral fica em torno de $20 - 10\log_{10}(500/3) \approx -2{,}2$ dBm em 3 kHz, abaixo do limite de 8 dBm. Por ser enquadrado no item 10.3, o sistema dispensa o salto de frequência e o limite de ocupação por canal exigidos para os canais de 125 kHz, o que permite usar um canal fixo. O módulo é homologado pela ANATEL (certificado nº 02021-18-07215), mas duas ressalvas se aplicam: a largura de faixa a 6 dB do sinal LoRa com 500 kHz nominais

não consta do manual e deve ser confirmada nos ensaios de homologação, e antenas com ganho superior a 6 dBi exigiriam a redução da potência (item 10.5).

O canal escolhido foi o de 915,9 MHz, o primeiro canal de 500 kHz do plano AU915: canal 64 na numeração da especificação (LoRa Alliance Regional Parameters Workgroup, 2025), que começa em zero, e canal 65 na numeração do manual do módulo, que começa em um. No módulo, ele foi selecionado pela máscara de canais (`AT+CHMASK`), e o *data rate* foi fixado em DR6 com o ajuste adaptativo de taxa (ADR) desativado, pois o comando `AT+DR` só tem efeito com o ADR desligado (Radioenge, 2022). A Tabela 12 reúne os parâmetros usados por todos os componentes do enlace.

Tabela 12 – Parâmetros do enlace LoRaWAN

Parâmetro	**Valor**
Plano de canais	AU915
Canal	Fixo, 915,9 MHz (canal 64), sem salto de frequência
Data rate	DR6: SF 8, BW de 500 kHz, ADR desativado
Camada física	CR 4/5, preâmbulo de 8 símbolos, CRC ativado, cabeçalho explícito, palavra de sincronismo `0x34`
Potência de transmissão	+20 dBm (fixa no módulo)
Ativação	ABP
Payload máximo em DR6	222 bytes
Portas de aplicação	FPort 1 (blocos de imagem) e FPort 2 (leitura do OCR)
Bloco de imagem	175 bytes de dados + 6 bytes de cabeçalho = 181 bytes
Intervalo entre pacotes	7 s

Fonte: Elaborado pelos autores (2026).

3.2.5.2 Formato das mensagens

Cada mensagem LoRaWan acrescenta 13 bytes ao conteúdo da aplicação: o cabeçalho MAC (1 byte), o endereço do dispositivo (4 bytes), o campo de controle (1 byte), o contador de quadros FCnt (2 bytes), a porta de aplicação FPort (1 byte) e o código de integridade MIC (4 bytes). O conteúdo da aplicação (FRMPayload) é criptografado com a chave de sessão de aplicação (AppSKey). Os dois fluxos usam portas diferentes, o que permite ao servidor distinguir as mensagens sem nenhum outro campo. A Tabela 13 apresenta o formato do conteúdo de cada porta.

Tabela 13 – Formato do *payload* de aplicação em cada fluxo

Campo	**Tamanho**	**Conteúdo**
FPort 1 — bloco de imagem (fluxo em nuvem)		
Identificador da leitura	2 bytes	Número aleatório sorteado a cada captura
Índice do bloco	2 bytes	Posição do bloco na imagem, a partir de 0
Total de blocos	2 bytes	Número de blocos da imagem
Dados	até 175 bytes	Trecho do arquivo JPEG
FPort 2 — leitura (fluxo em borda)		
Identificador da leitura	2 bytes	Número aleatório sorteado a cada leitura
Indicadores	1 byte	Bit 7: janela dos dígitos encontrada; bits 0 a 6: número N de dígitos
Dígitos	N bytes	Um caractere ASCII por rolete: `0` a `9`; `?` para dígito não classificado; `-` quando a janela não foi encontrada

Fonte: Elaborado pelos autores (2026).

Os campos numéricos são gravados em *big-endian*. O identificador da leitura impede que blocos de capturas diferentes sejam misturados na remontagem, e o par índice–total permite ao servidor saber quais blocos faltam. No fluxo em borda, com o medidor de seis roletes usado nos testes, a mensagem tem $3 + 6 = 9$ bytes e é enviada em um único *uplink*.

3.2.5.3 Preparo da imagem e fragmentação

No fluxo em nuvem, a câmera é configurada na resolução QQVGA (160×120 pixels) em tons de cinza, de modo que cada quadro bruto ocupa 19.200 bytes (um byte por pixel). Esse quadro é comprimido em JPEG pelo próprio ESP32-CAM, por software, com qualidade 90 em uma escala de 1 a 100.

A escolha do formato foi apoiada por uma prova de conceito em C++ com a biblioteca OpenCV, que mediu o tamanho de três imagens de hidrômetro em 160×120 pixels após a compressão JPEG em diferentes qualidades, em três versões: colorida, em tons de cinza e binarizada pelo método de Otsu. A Tabela 14 mostra as médias obtidas para algumas qualidades.

Tabela 14 – Tamanho médio do JPEG de três imagens de 160×120 pixels

Qualidade JPEG	**Colorida (bytes)**	**Tons de cinza (bytes)**	**Binarizada (bytes)**
30	2.028	1.626	2.773
50	2.338	1.949	3.433
80	2.910	2.518	4.725
90	3.466	3.073	5.895

Fonte: Elaborado pelos autores (2026).

A conversão para tons de cinza reduziu o arquivo em relação à imagem colorida, enquanto a binarização o aumentou em quase o dobro, pois as bordas abruptas entre preto e branco são pouco compressíveis pelo JPEG. Por isso, a imagem é transmitida em tons de cinza, sem binarização. A qualidade 90 foi mantida para que a compressão não prejudicasse o reconhecimento dos dígitos no servidor. Nas 60 leituras das duas corridas de medição do fluxo em nuvem, o JPEG variou de 3.323 a 4.568 bytes (mediana de 4.323 bytes), pois o tamanho depende da cena e da iluminação. Para o dimensionamento, adotou-se como referência uma imagem de 4.300 bytes, valor definido a partir das primeiras capturas, antes das corridas de medição, e próximo da mediana observada depois.

O arquivo JPEG é dividido em blocos de 175 bytes, que recebem o cabeçalho de 6 bytes da Tabela 13, totalizando 181 bytes de *payload*, abaixo do máximo de 222 bytes permitido em DR6. Uma versão anterior usava blocos de 100 bytes; o aumento para 175 bytes reduziu o número de pacotes por imagem a pouco mais da metade, mantendo uma folga de 41 bytes em relação ao máximo.

Os blocos são enviados ao módulo de rádio, um a um, pelo comando `AT+SENDB` na porta 1, com intervalo de 7 s entre eles. O item 10.3 não impõe intervalo mínimo entre transmissões em um canal fixo de 500 kHz; o valor de 7 s é conservador e foi mantido para não sobrecarregar o módulo. Se o módulo não confirmar a transmissão em até 2 s, o bloco é reenviado, até cinco tentativas.

3.2.5.4 Tempo no ar

O tempo no ar de cada pacote foi calculado pela fórmula da Semtech para o modem LoRa (Semtech Corporation, 2013). A duração de um símbolo é dada pela Equação 3.4:

$$T_{\text{sym}} = \frac{2^{SF}}{BW} = \frac{2^8}{500\,000\ \text{Hz}} = 0{,}512\ \text{ms} \tag{3.4}$$

O número de símbolos do *payload* é dado pela Equação 3.5:

$$n_{\text{payload}} = 8 + \max\left(\left\lceil \frac{8 \cdot PL - 4 \cdot SF + 28 + 16 \cdot CRC - 20 \cdot H}{4 \cdot (SF - 2 \cdot DE)} \right\rceil \cdot (CR + 4),\ 0\right) \tag{3.5}$$

em que PL é o tamanho do pacote na camada física, em bytes; $CRC = 1$ indica CRC ativado; $H = 0$ indica cabeçalho explícito; $DE = 0$ indica a otimização para baixas taxas desativada; e $CR = 1$ corresponde ao código 4/5. O tempo no ar do pacote é dado pela Equação 3.6, com preâmbulo de $n_{\text{pre}} = 8$ símbolos:

$$T_{\text{pacote}} = (n_{\text{pre}} + 4{,}25 + n_{\text{payload}}) \cdot T_{\text{sym}} \tag{3.6}$$

Para um bloco de imagem completo, $PL = 13 + 181 = 194$ bytes, o que resulta em $n_{\text{payload}} = 8 + \lceil 1564/32 \rceil \cdot 5 = 253$ símbolos e $T_{\text{pacote}} = (8 + 4{,}25 + 253) \cdot 0{,}512 \approx 135{,}8$ ms. Para a mensagem de leitura do fluxo em borda, $PL = 13 + 9 = 22$ bytes, o que resulta

em $T_{\text{pacote}} \approx 25{,}7$ ms. A Tabela 15 apresenta o número de pacotes e os tempos para os tamanhos mínimo, mediano e máximo de JPEG medidos e para a mensagem do fluxo em borda.

Tabela 15 – Pacotes, tempo de rádio e tempo total de envio por leitura

Conteúdo	**Tamanho (bytes)**	**Pacotes**	**Rádio (s)**	**Envio total (s)**
JPEG, menor medido	3.323	19	2,58	126
JPEG, mediana medida	4.323	25	3,36	168
JPEG, maior medido	4.568	27	3,57	182
Leitura do fluxo em borda	9	1	0,03	0,03

Fonte: Elaborado pelos autores (2026). O envio total soma o tempo no ar ao intervalo de 7 s entre pacotes.

Os valores mostram que, no fluxo em nuvem, o tempo de rádio é pequeno diante do tempo total de envio, que é dominado pelos intervalos de 7 s entre os pacotes. O volume transmitido por leitura também difere em mais de duas ordens de grandeza entre os fluxos: cerca de 4 KB de imagem contra 9 bytes de leitura. Essa diferença é a base das hipóteses H2 e H3.

3.2.5.5 Recepção, descriptografia e remontagem

O receptor LoRa32 foi configurado com os mesmos parâmetros de camada física do transmissor (Tabela 12). Para cada quadro recebido com CRC válido, ele escreve na porta serial uma linha com o conteúdo do quadro em hexadecimal e o RSSI medido. Como não há servidor de rede, o processamento LoRaWan é feito no computador pelo *script* `reconstrutor.py`, em quatro etapas:

1. **Separação dos campos**: o quadro é dividido em endereço do dispositivo, contador FCnt, porta e conteúdo; quadros de outros endereços são descartados.

2. **Descriptografia**: o conteúdo é descriptografado com a AppSKey, usando o AES-128 em modo contador definido pela especificação LoRaWan, que combina o endereço do dispositivo e o FCnt na geração do fluxo de chaves. O MIC não é verificado, pois o computador não dispõe da chave de rede.

3. **Tratamento por porta**: na porta 2, a leitura é decodificada e validada (tamanho coerente com o número de dígitos e apenas caracteres permitidos). Na porta 1, os blocos são agrupados pelo identificador da leitura, e blocos repetidos são ignorados.

4. **Remontagem e registro**: quando todos os blocos de uma leitura chegam, o arquivo JPEG é gravado e pode seguir para o OCR. Se algum bloco faltar, a imagem é gravada como parcial, com zeros no lugar dos dados ausentes. Cada leitura é registrada em um arquivo CSV com os blocos recebidos, o RSSI e os tempos usados nas medições.

No modo autônomo, usado nas medições, nenhum dos dois firmwares retransmite mensagens perdidas: o nó envia a leitura ou a imagem uma única vez e volta a dormir. O firmware do fluxo em nuvem aceita um comando de reenvio de blocos específicos, mas ele só foi usado em testes manuais. Assim, no fluxo em nuvem, a perda de um único bloco inutiliza a imagem inteira, e, no fluxo em borda, a perda do único pacote inutiliza a leitura. Esse comportamento é considerado na análise da taxa de entrega.

3.2.6 Reconhecimento dos dígitos

O OCR implementado divide a leitura em duas etapas: primeiro, localizar no quadro a janela onde ficam os roletes do hidrômetro; depois, classificar cada dígito dessa janela separadamente. Os mesmos modelos e a mesma lógica de leitura são usados no ESP32-CAM, no fluxo em borda, e no servidor, no fluxo em nuvem. Esta seção descreve como essa solução foi escolhida, os dados usados, os dois modelos, a quantização e a forma de leitura adotada.

3.2.6.1 Evolução das abordagens

3.2.6.1.1 Teste de viabilidade com OCR genérico.

O primeiro experimento verificou se um OCR de propósito geral conseguiria ler o mostrador em uma imagem tão pequena quanto a capturada pelo ESP32-CAM (160×120 pixels, em tons de cinza). Em um *notebook* no Google Colab, o Tesseract 4.1.1 e o EasyOCR 1.7.2 foram aplicados a uma foto do mostrador, cujo valor visível era `353485`. Aplicados à linha inteira, nenhum dos dois leu o valor: o Tesseract devolveu sequências sem sentido, como `flslalalals`, e o EasyOCR devolveu `1581`. A leitura só foi correta quando a imagem foi separada em um recorte por dígito, por análise de componentes conectados com o OpenCV, e cada recorte foi reconhecido isoladamente. Por usar uma única foto, o teste serviu apenas como prova de viabilidade, mas indicou a estrutura adotada: localizar os dígitos e classificá-los um a um. Os OCR genéricos não foram usados nas etapas seguintes.

3.2.6.1.2 Pipelines avaliados.

A partir dessa estrutura, três pipelines de OCR foram desenvolvidos em paralelo, e apenas um foi mantido. A Tabela 16 resume os três.

Tabela 16 – Pipelines de OCR desenvolvidos

Pipeline	**Localização da janela**	**Leitura dos dígitos**	**Situação**
A	U-Net de 384 × 384 pixels, treinada com QAT	CRNN com CTC, lendo a janela inteira de uma vez (48 × 224 pixels)	Descartado
B	Rede codificador–decodificador de 192 × 192 pixels, INT8	CNN de um dígito (28 × 28 pixels) aplicada a cada rolete	**Adotado**
C	A mesma U-Net do pipeline A	CNN de um dígito (68 × 68 pixels), treinada com dígitos de fontes genéricas	Descartado

Fonte: Elaborado pelos autores (2026).

O pipeline A aplicava a arquitetura CRNN com função de perda CTC, apresentada na Seção 2.2.5.5, e foi testado diretamente no ESP32-CAM. O teste mostrou que ele não pode ser executado no microcontrolador: o modelo de localização exigiu 15.335.424 bytes de memória de trabalho (*arena*) do TensorFlow Lite Micro, 3,8 vezes os cerca de 4 MB de PSRAM da placa, e os modelos de leitura usam o operador `REDUCE_PROD`, ausente na versão da biblioteca disponível no *core* Arduino-ESP32 3.0.7. O descarte vale para os modelos como foram exportados e não invalida a abordagem com CTC, que permanece como possibilidade de trabalho futuro. O pipeline C foi descartado por rodar apenas no computador e por ter o classificador treinado com dígitos impressos e manuscritos, e não com imagens de hidrômetros. O pipeline B, adotado, é descrito a seguir.

3.2.6.2 Conjuntos de dados

A principal dificuldade para treinar um classificador de dígitos de hidrômetro é a rotulagem: os conjuntos públicos informam a leitura completa do medidor, e não o valor de cada rolete. Três fontes de dados foram combinadas, cada uma com um tratamento próprio para gerar recortes rotulados de um único dígito:

1. **Water Meters Dataset** (KUCEV, 2020): 1.244 fotos de hidrômetros variados, tiradas em ângulos arbitrários, com a máscara da janela de dígitos e a leitura no nome do arquivo. Foi usado no treinamento do modelo de localização e, para o classificador, passou por um processo automático: a janela foi retificada a partir da máscara e dividida em células por projeção vertical. Para evitar rótulos errados, só foram aceitas as fotos em que a divisão produziu exatamente oito células de larguras semelhantes; as demais foram descartadas. Restaram 195 fotos, que geraram 1.560 recortes.

2. **Word-Wheel Water Meter Dataset** (ZHAO et al., 2026): conjunto público cujo subconjunto de reconhecimento traz recortes padronizados de 200×60 pixels da janela

de dígitos, com a leitura completa. Foram descartadas as imagens com um rolete no meio da troca entre dois dígitos, cujo rótulo é ambíguo, e as imagens espelhadas foram invertidas. Como os recortes já vêm retos, cada um foi dividido em fatias de largura igual. Apenas as pastas de treino foram usadas, gerando 47.478 recortes; as pastas de teste, com 1.894 imagens, foram reservadas como teste independente.

3. **Fotos do próprio ESP32-CAM**: para aproximar o modelo das condições do sensor, da lente e da compressão do dispositivo, três fotos capturadas pelo ESP32-CAM, de dois medidores diferentes, tiveram a janela marcada manualmente e foram fatiadas e rotuladas à mão, gerando 24 recortes. Esses recortes foram usados apenas no ajuste fino (*fine-tuning*) do classificador.

Os dois conjuntos públicos somam 49.038 recortes, com forte desbalanceamento: o dígito 0 tem 20.398 exemplos, pois os roletes mais significativos do medidor costumam marcar zero, enquanto os demais dígitos têm entre 2.469 e 4.991. Para a validação, a divisão foi feita por foto, e não por recorte, para que dígitos de uma mesma foto não aparecessem ao mesmo tempo no treino e na validação: 15% das fotos foram separadas, resultando em 41.660 recortes de treino e 7.378 de validação.

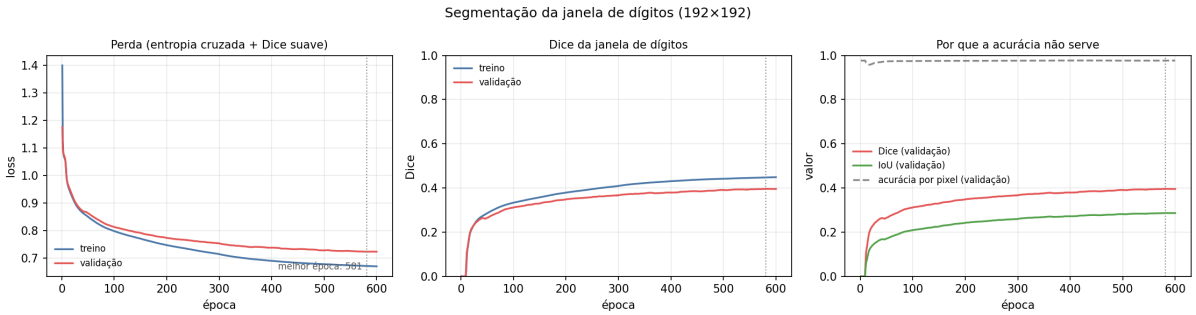
Uma limitação importante deve ser registrada: as fotos capturadas pelo ESP32-CAM neste trabalho não são de um hidrômetro físico, mas de um monitor que exibia imagens de hidrômetros. Elas reproduzem o sensor, a lente e a compressão da câmera, mas não os reflexos no vidro, a iluminação e o ângulo de um medidor instalado.

3.2.6.3 Modelo de localização da janela

O modelo de localização recebe o quadro redimensionado para 192×192 pixels e produz uma máscara binária que indica os pixels da janela de dígitos. Trata-se de uma rede codificador–decodificador inspirada na U-Net, porém reduzida para caber no microcontrolador e sem as conexões de salto: duas camadas convolucionais (3×3, com 8 e 16 filtros), cada uma seguida de *max pooling*, duas convoluções transpostas que restauram a resolução e uma convolução 1×1 com ativação sigmoide na saída. Quantizada em INT8, ocupa 11,8 KB.

O modelo foi treinado com o *Water Meters Dataset*, dividido em 70% para treino, 15% para validação e 15% para teste. Como a janela ocupa uma fração pequena da imagem, a entropia cruzada sozinha levava a rede a classificar quase tudo como fundo; por isso, a perda adotada combina entropia cruzada e Dice suave, o que elevou o Dice de validação de 0,306 para 0,396 com o mesmo número de épocas. O treino durou 600 épocas, com lotes de 16 imagens, e a melhor época foi a 581. A Figura 8 mostra a evolução do treino.

Figura 8 – Evolução do treino do modelo de localização da janela



Fonte: Elaborado pelos autores (2026).

No conjunto de teste, com 186 fotos não vistas no treino, o modelo em INT8 obteve Dice de 0,39 e IoU de 0,28. O resultado é fraco. A acurácia por pixel supera 98%, mas não serve como medida, pois o fundo domina a imagem, como mostra o terceiro gráfico da Figura 8. As curvas de treino e validação próximas indicam subajuste: a rede, minúscula e sem conexões de salto, não tem capacidade para localizar a janela com precisão.

No uso, a caixa da janela é o menor retângulo que contém os pixels marcados pela máscara, convertido para as coordenadas do quadro de 160 × 120 pixels. Como a máscara prevista transborda a janela, essa caixa fica muito maior do que deveria. Nas fotos do conjunto de teste comum, descrito adiante, a caixa prevista teve mediana de 125 × 85 pixels, cobrindo quase todo o mostrador, enquanto a janela real mede cerca de 97 × 18 pixels. Com essa caixa, cada fatia contém pedaços de vários roletes e do mostrador, e a leitura falha.

3.2.6.4 Classificador de dígitos

O classificador recebe a imagem de um único rolete, redimensionada para 28 × 28 pixels em tons de cinza, e devolve a probabilidade de cada um dos dez dígitos. É uma rede neural convolucional com 72.442 parâmetros, cuja arquitetura é apresentada na Tabela 17. Quantizado em INT8, o modelo ocupa 81,5 KB.

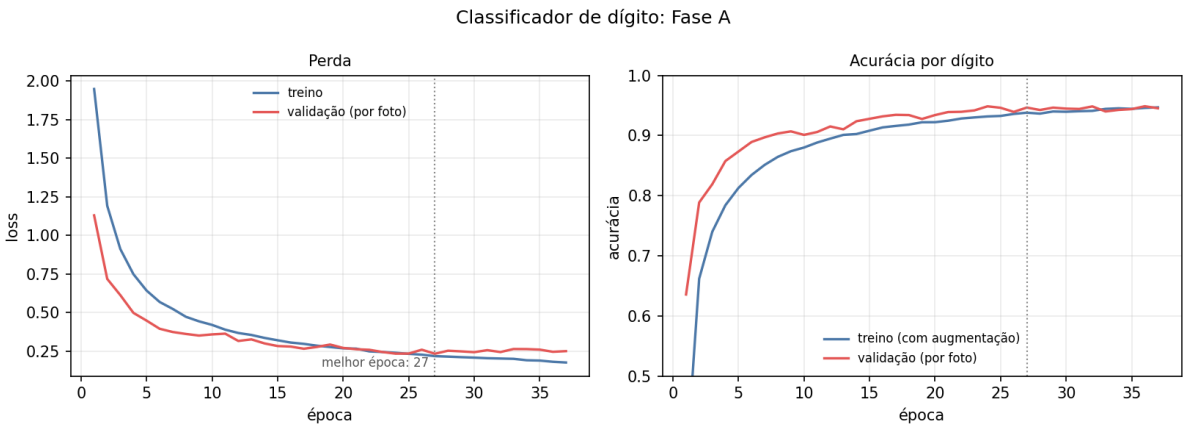
Tabela 17 – Arquitetura do classificador de dígitos

Camada	**Configuração**	**Saída**
Entrada	imagem em tons de cinza	$28 \times 28 \times 1$
Convolução $\times 2$	16 filtros 3×3, ReLU	$28 \times 28 \times 16$
Max pooling	2×2	$14 \times 14 \times 16$
Convolução $\times 2$	32 filtros 3×3, ReLU	$14 \times 14 \times 32$
Max pooling	2×2	$7 \times 7 \times 32$
Convolução	64 filtros 3×3, ReLU	$7 \times 7 \times 64$
Max pooling	2×2	$3 \times 3 \times 64$
Achatamento	—	576
Densa	64 neurônios, ReLU, *dropout* de 30%	64
Densa (saída)	10 neurônios, *softmax*	10

Fonte: Elaborado pelos autores (2026).

O treinamento foi feito em duas fases. Na fase A, o modelo foi treinado com os recortes dos dois conjuntos públicos, com o otimizador Adam (taxa de aprendizado de 5×10^{-4}), lotes de 128 recortes e pesos por classe para compensar o excesso de zeros. Para aproximar os recortes das imagens do ESP32-CAM, foi aplicado aumento de dados com redução e ampliação da resolução, ruído gaussiano, pequenas rotações e translações e variação de contraste. O treino foi interrompido por parada antecipada após 37 épocas, com a melhor época na 27.ª, e atingiu 94,65% de acurácia na validação (Figura 9).

Figura 9 – Evolução do treino do classificador de dígitos (fase A)



Fonte: Elaborado pelos autores (2026).

Na fase B, o modelo passou por um ajuste fino de cinco épocas, com taxa de aprendizado dez vezes menor (5×10^{-5}), usando os 24 recortes do ESP32-CAM, fortemente aumentados e repetidos, junto com uma parte dos recortes da fase A, para que o modelo não esquecesse o que já havia aprendido. Ao final, a acurácia de validação do modelo em ponto flutuante foi de 95,46%.

3.2.6.5 Quantização e exportação

Os dois modelos foram quantizados em INT8 após o treinamento (*post-training quantization*), com um conjunto representativo de imagens para calibrar as escalas. A quantização foi completa: pesos, ativações, entrada e saída em inteiros de 8 bits, usando apenas operadores INT8 do TensorFlow Lite. Na entrada, a escala de $1/255$ com ponto zero de -128 permite converter cada pixel diretamente, subtraindo 128 do seu valor. Diferente do pipeline A, que usou treinamento consciente da quantização (QAT), o pipeline B não precisou dessa técnica: a perda de acurácia na quantização foi desprezível, como mostra a avaliação da hipótese H1 nos resultados. Por fim, cada modelo `.tflite` foi convertido em um vetor de bytes em um arquivo de cabeçalho C (`.h`), incluído no firmware.

3.2.6.6 Leitura com janela calibrada na instalação

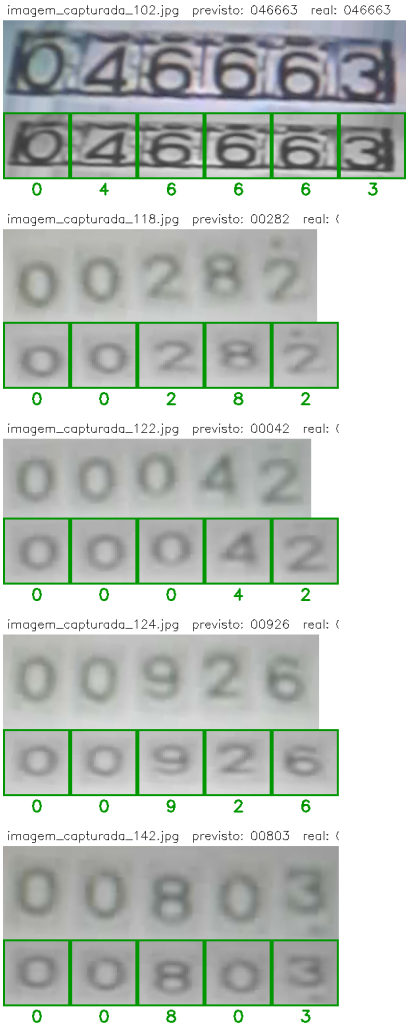
Dada a caixa da janela, a leitura é feita da mesma forma no firmware e no servidor: a caixa é dividida em N fatias de largura igual, uma por rolete; cada fatia é redimensionada para 28×28 pixels por vizinho mais próximo, convertida para INT8 e classificada; e os dígitos de maior probabilidade são concatenados na leitura. O número N de roletes é um parâmetro configurado por medidor: seis no medidor usado nos testes de hardware.

Como o modelo de localização não encontra a janela com precisão, os resultados com a caixa prevista por ele são próximos de zero. Esses resultados **não refletem a capacidade de leitura do classificador**, e sim o erro da localização: com a caixa errada, as fatias não correspondem aos roletes. Quando a janela é informada corretamente, o mesmo classificador lê os dígitos com alta acurácia. A Figura 10 mostra cinco capturas do ESP32-CAM em que a janela foi marcada à mão: todas as cinco leituras e os 26 dígitos foram reconhecidos corretamente.

Por isso, a solução adotada dispensa a localização automática. Em uma instalação real, a câmera fica fixa diante do medidor, de modo que a janela de dígitos ocupa sempre a mesma região do quadro. Assim, a região de interesse (ROI) e a inclinação dos roletes são calibradas uma única vez, na instalação, e usadas em todas as leituras seguintes. Essa calibração precisa ser precisa: a avaliação apresentada nos resultados indica que a ROI deve estar a cerca de um ou dois pixels da posição correta no quadro de 160×120 pixels, em que cada dígito tem aproximadamente 16 pixels de largura.

Nas medições de latência e consumo em hardware, o firmware de borda ainda executava o modelo de localização a cada leitura, com a câmera posicionada à mão para que a janela ocupasse a maior parte do quadro e a caixa prevista ficasse próxima dela. Essa configuração é descrita na próxima seção, e o efeito de remover a localização é discutido nos resultados.

Figura 10 – Leitura com a janela de dígitos marcada manualmente em cinco capturas do ESP32-CAM



Fonte: Elaborado pelos autores (2026). Em cada captura, a faixa superior é a janela recortada, a inferior mostra as fatias e, abaixo, o dígito previsto para cada uma.

3.3 Experimentos e cenário de testes (opcional)

Esta seção é **opcional** e deve ser incluída quando o trabalho prevê avaliação experimental. Descreva os cenários planejados, os parâmetros testados e por que cada experimento está alinhado aos objetivos. Liste os conjuntos de dados, as variações e o número de repetições.

3.4 Resultados e análises

Apresente os resultados com tabelas, gráficos e figuras e, sobretudo, **analise-os** à luz da revisão bibliográfica e da hipótese — resultados devem ser analisados, não apenas exibidos. Diferencie:

- **Figura:** imagens, diagramas conceituais ou arquiteturais que ilustram estrutura ou

fluxo.

- **Gráfico:** plot de dados (linhas, barras, boxplots) que compara métricas obtidas.

Exemplo de tabela:

Tabela 18 – Comparação de métricas reportadas

Método	Acurácia	Tempo médio (ms)
Proposta	92%	180
Baseline	88%	220

Fonte: Elaborado pelo autor (2025)

Ao inserir um gráfico (por exemplo `assets/figures/desempenho.png`), use — lembrando que, pela ABNT, o `\caption` (título) vem **acima** da imagem e a fonte (`\legend`) vem **abaixo**:

```
\begin{figure}[htb]
  \centering
  \caption{Gráfico de comparação das métricas de acurácia.}
  \label{fig:grafico-desempenho}
  \includegraphics[width=0.8\textwidth]{assets/figures/desempenho.png}
  \legend{Fonte: Elaborado pelo autor (2025)}
\end{figure}
```

Na sequência, faça a **análise crítica e comparação**, discutindo as descobertas em relação aos trabalhos correlatos e evidenciando ganhos e perdas; por fim, na **discussão**, relacione os resultados aos objetivos e à hipótese, explicitando se o critério de sucesso foi atingido.

O desenvolvimento só é pesquisa se estiver claramente ligado ao problema e à metodologia definida.

4 Conclusão

4.1 Retomada do problema e dos objetivos

Retome o problema e destaque o que foi atingido.

4.2 Contribuições para a Computação

Explique o que a pesquisa trouxe de novo.

4.3 Limitações e trabalhos futuros

Liste limites e sugira próximos passos coerentes.

Observações importantes

Para graduação, Wazlawick aceita trabalhos aplicados desde que:

- O problema seja preciso;
- A solução esteja fundamentada;
- Haja análise crítica;
- Não seja apenas entrega funcional.

Apresentação oral (banca)

Além do documento escrito, o TCC2 é avaliado por uma **apresentação oral** final perante a banca. Um template pronto em `Beamer` está disponível no arquivo `apresentacao_tcc2.tex`, na mesma pasta deste documento.

Regras da apresentação do TCC2

- **Duração: 30 minutos.** Cronometre os ensaios; ultrapassar o tempo é penalizado. A arguição da banca ($\approx$ 5 min) ocorre após a apresentação, separada desse total.
- **Cubra o trabalho completo**, seguindo a estrutura do documento escrito: introdução e problema, objetivos e método, desenvolvimento, **resultados e análise** e conclusão/contribuições.
- **Dê peso aos resultados:** grande parte do tempo deve ser dedicada a apresentar e **analisar criticamente** os resultados, não apenas descrevê-los.
- **Ritmo:** cerca de 1 a 1,5 minuto por slide, totalizando aproximadamente 20 a 26 slides de conteúdo.
- **Logo do SENAC:** grande no primeiro slide (capa) e pequena no canto superior direito dos demais slides — já configurado no template.
- **Demonstração:** se houver demo ao vivo, tenha um plano B com capturas de tela ou vídeo curto.
- **Slides de apoio (backup):** inclua slides extras após o "Obrigado" com detalhes técnicos, trechos de código e tabelas completas para a arguição.

Distribuição de tempo sugerida (30 min): introdução/problema $\approx$ 4 min; objetivos/método $\approx$ 5 min; desenvolvimento $\approx$ 8 min; resultados/análise $\approx$ 8 min; conclusão $\approx$ 3 min; demonstração $\approx$ 2 min.

Glossário

Cloud Computing Método de processamento em nuvem 2, 3, 11, 23, 24, 39

Edge Computing Método de processamento local 2, 3, 11, 12, 23, 24, 39

ESP32-CAM Módulo para acoplamento no microprocessador ESP32, conferindo ao dispositivo a capacidade de capturar imagens 2–4, 12, 13, 15, 37, 39–41, 45, 48–50, 52–54

payload Caracteres enviados no corpo do pacote de rede 18, 22

retrofitting Técnica de modernização ou atualização de dispositivos e equipamentos 2, 3, 12, 13

Smart Metering Tecnologias para Medição Inteligente 2, 3, 12

Referências

ADAM, L. *Desenvolvimento e implantação de solução para comunicação bidirecional de longo alcance com medidores de energia elétrica*. Dissertação (Dissertação (Mestrado em Engenharia Elétrica)) — Universidade Federal de Santa Maria, Santa Maria, RS, 2022. Programa de Pós-Graduação em Engenharia Elétrica, Área de Concentração em Sistemas Elétricos de Potência. 11, 25

ADEMOLA, O. A.; PETLENKOV, E.; LEIER, M. Resource-aware scene text recognition using learned features, quantization, and contour-based character extraction. *IEEE Access*, v. 11, p. 56865–56874, 2023. Revista de acesso aberto (IEEE Access). 35

AHMAD, S. et al. Comparative analysis and practical implementation of the esp32 microcontroller module for the internet of things. In: *2021 International Conference on Computing, Electronic and Electrical Engineering (ICE Cube)*. [S.l.: s.n.], 2021. p. 1–6. 20

ALVISI, S. et al. Wireless middleware solutions for smart water metering. *Sensors*, v. 19, n. 8, 2019. 12

ANATEL. *Ato nº 14.448, de 4 de dezembro de 2017. Aprova os Requisitos Técnicos para a Avaliação da Conformidade de Equipamentos de Radiocomunicação de Radiação Restrita*. 2017. Boletim de Serviço Eletrônico da Anatel, Brasília, 02 jan. 2018. Acesso em: 16/09/2026. Disponível em: <https://informacoes.anatel.gov.br/legislacao/atos-de-certificacao-de-produtos/2017/1139-ato-14448>. 24, 25, 43

ANDRIULO, F. C. et al. Edge computing and cloud computing for internet of things: A review. *Informatics*, MDPI, v. 11, n. 4, p. 71, 2024. Disponível em: <https://doi.org/10.3390/informatics11040071>. 12, 13, 23, 24

ATIKA, L. et al. Image segmentation for mitral regurgitation with convolutional neural network based on unet, resnet, vnet, fractalnet and segnet: A preliminary study. *Big Data and Cognitive Computing*, MDPI, v. 6, n. 4, p. 141, 2022. Disponível em: <https://doi.org/10.3390/bdcc6040141>. 31, 33

AYENI, J. A. Convolutional neural network (cnn): The architecture and applications. *Applied Journal of Physical Science*, Integrity Research Journals, v. 4, n. 4, p. 42–50, 2022. ISSN 2756-6684. 31, 32

BARKAT, S. *Smart Water Management System Using Deep Learning and IoT-Based Predictive Modeling*. Master's Thesis — Department of Computer Science, 2024. 38

BRAVO, J. K. G.; JAVIER, C. P.; TOLENTINO, V. T. Ants: Automated network tracking system for water consumption using lora technology. *FAITH Colleges*, 2025. Acesso em: 23 abr. 2026. 35, 37, 38

CHEN, X. et al. Text recognition in the wild: A survey. *ACM Computing Surveys*, v. 54, n. 2, p. 1–35, 2021. Versão de acesso aberto: arXiv:2005.03492. 32, 33

FERREIRA, A.; CAVALCANTE, A. L. de C. Iot on an esp32: Optimization methods regarding battery life and write speed to an sd-card. *Journal of Software Engineering and Applications*, v. 13, n. 9, 2020. 20

FERREIRA, G. B. *Compressão de redes neurais artificiais com restrições de acurácia utilizando otimização multiobjetivo baseada em modelos substitutos*. 38 p. Monografia (Monografia (Bacharelado em Ciência da Computação)) — Universidade Federal de Ouro Preto, Instituto de Ciências Exatas e Biológicas, Departamento de Computação, Ouro Preto, MG, 2022. 35

Fortune Business Insights. *Smart Water Metering Market Size, Share & Industry Analysis*. [S.l.], 2025. Disponível em: <https://www.fortunebusinessinsights.com/pt/industry-reports/smart-water-metering-market-100776>. Acesso em: 24 abr. 2026. 11

GRAVES, A. et al. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In: *Proceedings of the 23rd International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2006. p. 369–376. PDF público: <https://www.cs.toronto.edu/~graves/icml_2006.pdf>. 34, 35

HANNUN, A. Sequence modeling with ctc. *Distill*, v. 2, n. 11, p. e8, 2017. 30, 33, 34

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 1997. PDF de acesso aberto disponível em: <https://direct.mit.edu/neco/article-pdf/9/8/1735/813796/neco.1997.9.8.1735.pdf>. 29, 30

JAWAS, N.; INDRIANTO. Image based automatic water meter reader. *Journal of Physics: Conference Series*, v. 953, p. 012027, 2018. Artigo de acesso aberto (IOP Conference Series). 32, 35

JESUS, V. A. de; COQUEIRO, T. A. S. Inteligência artificial: Uma breve revisão bibliográfica de ferramentas de machine learning e deep learning na medicina. *Faculdade de Computação, Universidade Federal do Pará (UFPA)*, Castanhal, PA, Brasil, 2023. Trabalho de Conclusão / Artigo Acadêmico. 29, 30, 31

KUCEV, R. *Water Meters Dataset*. Kaggle, 2020. Disponível em: <https://www.kaggle.com/datasets/tapakah68/yandextoloka-water-meters-dataset>. Acesso em: 02 jun. 2026. 49

LoRa Alliance Regional Parameters Workgroup. *LoRaWAN Regional Parameters RP002-1.0.5*. [S.l.], 2025. Authored by the Technical Committee Regional Parameters Workgroup. Editor: D. Tholl. Disponível em: <https://lora-alliance.org/>. 18, 25, 26, 27, 28, 44

NAZRE, R. et al. A temporal convolutional network-based approach for network intrusion detection. In: *arXiv preprint arXiv:2412.17452*. [S.l.: s.n.], 2024. PDF de acesso aberto: <http://export.arxiv.org/pdf/2412.17452v1>. 30

NGUYEN, V. K. et al. Solution for direct implementation of recurrent neural networks on microcontrollers. *Journal of Science and Technology*, 2026. Artigo de acesso aberto. Disponível em: <https://jst.tnu.edu.vn/jst/article/view/15162>. 30

PASTóRIO, A. F. et al. Fundamentos de lorawan – teoria e prática. *Federal University of Technology of Paraná*, 2022. 16, 18, 20, 21, 22

Radioenge. *EndDevice LoRaWAN Radioenge: manual de operação*. Revisão de junho de 2022. [S.l.], 2022. 43, 44

ROBINSON, S. *StuartCAM – ESP32CAM Picture Transfers with LoRa*. 2021. Disponível em: <https://stuartsprojects.github.io/2021/11/21/StuartCAM-ESP32CAM-Picture-Transfers-with-LoRa.html>. 21

SARI, M. E. et al. Design and implementation of esp32-based iot devices. *Sinkron: Jurnal dan Penelitian Teknik Informatika*, v. 7, n. 1, 2022. 20

Semtech Corporation. *AN1200.13: SX1272/3/6/7/8 LoRa Modem Design Guide*. [S.l.], 2013. Disponível em: <https://www.semtech.com>. 46

Semtech Corporation. *AN1200.86: LoRa and LoRaWAN*. [S.l.], 2024. Application Note Version 1.0. Disponível em: <https://www.semtech.com>. 19, 21, 22, 26

Semtech Corporation. *AN1200.87: LoRaWAN Device Classes*. [S.l.], 2024. Application Note Version 2.0. Disponível em: <https://www.semtech.com>. 16

Semtech Corporation. *AN1200.88: Sending and Receiving Messages*. [S.l.], 2024. Application Note Version 2.0. Disponível em: <https://www.semtech.com>. 17

SHI, B.; BAI, X.; YAO, C. An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, v. 39, n. 11, p. 2298–2304, 2017. Versão de acesso aberto: arXiv:1507.05717. 30, 33, 34, 35

TEDJOJUWONO, S. M.; JAHJA, J. D. Smart water management system using IOT based sensors. In: *Proceedings of the 7th International Conference on Eco Engineering Development 2023 (ICEED 2023)*. [S.l.]: IOP Publishing, 2024. (IOP Conference Series: Earth and Environmental Science, 012106), p. 1–9. 11, 35, 36

UGGERI, D.; SIMEON, E. *Smart water metering: analytical model computation for an estimated costs and benefits analysis*. Dissertação (Mestrado) — Politecnico di Milano, 2020. Disponível em: <https://www.politesi.polimi.it/handle/10589/171225>. Acesso em: 20 abr. 2026. 11

VILELA, R. da S. Solução iot para digitalização de medidores de consumo por visão computacional e microml. *Universidade Federal de Campina Grande*, 2021. 11, 12, 28, 29, 35

ZHAO, S. et al. A comprehensive dataset for word-wheel water meter reading under challenging conditions. *Scientific Data*, v. 13, p. 479, 2026. 49